Proyecto MetaU - Propuesta Nuevo Avatar

Documento de Análisis y Diseño

Estado actual: Primer Sprint – Microservicios REST

Programación V

Colegio Universitario de Cartago

Prof. Ing. Gabriel González Solano

Grupo: Akatsukis

Sebastián Jiménez Arrieta
Brandon Steve Guido Navarro
Sebastián Obando Ramírez
Julián Solano Obando
Anthony Emmanuel Gamboa Elizondo

Cartago, Costa Rica
21 de octubre de 2026

Tabla de contenido

1. Introducción 3
2. Arquitectura general del sistema 4
4. Diagramas de casos de uso 5
4.1 Caso de Uso de la Arquitectura 5
4.2 Caso de Uso del Microservicio Usuarios 5
4.3 Caso de Uso del Microservicio Roles 6
4.4 Caso de Uso del Microservicio Parámetros 6
4.5 Caso de Uso del Microservicio Módulos 7
4.6 Caso de Uso del Microservicio Login 7
4.7 Caso de Uso del Microservicio Instituciones 8
4.7 Caso de Uso del Microservicio Carreras 8
4.8 Caso de Uso del Microservicio Cursos 9
4.9 Caso de Uso del Microservicio Grupos 9
4.10 Caso de Uso del Microservicio Periodos 10
4.11 Caso de Uso del Microservicio Profesores 10
4.12 Caso de Uso del Microservicio Pre-Matriculas 11
4.13 Caso de Uso del Microservicio Matriculas 11
4.14 Caso de Uso del Microservicio Expediente 12
4.15 Caso de Uso del Microservicio Direcciones 12
4.16 Caso de Uso del Microservicio Notas 13
4.17 Caso de Uso del Microservicio Historial Academico 13
4.18 Caso de Uso del Microservicio Listado Estudiantes 13
4.19 Caso de Uso del Microservicio Facturas 14
4.20 Caso de Uso del Microservicio Pagos 14
4.21 Caso de Uso del Microservicio Notificaciones 15
4.22 Caso de Uso del Microservicio Bitácoras 15
4.23 Caso de Uso General del Website Administrativo 16
5. Diagramas de clases 17
5.1 USR1 – Administración de usuarios 17
5.2 USR2 – Administración de roles 18
5.3 USR3 – Administración de parámetros 19
5.4 USR4 – Administración de módulos 20
5.5 USR5 – Login y manejo de tokens 21

5.6 ACD1 – Administración de instituciones 22
5.7 ACD2 – Administración de carreras 23
5.8 ACD3 – Administración de cursos 24
5.9 ACD4 – Administración de grupos 25
5.10 ACD5 – Administración de periodos 26
5.11 ACD6 – Administración de profesores 26
5.12 MAT1 – Administración de prematricula 27
5.13 MAT2 – Administración de matrícula 28
5.14 MAT3 – Administración de expedientes de estudiantes 29
5.15 MAT4 – Administración de direcciones 30
5.16 MAT5 – Administración de notas 31
5.17 ACA1 – Consulta del historial académico 32
5.18 ACA2 – Consulta de listado de estudiantes 32
5.19 IPN1 – Administración de la facturación 33
5.20 IPN2 – Administración de los pagos 34
5.21 IPN3 – Gestión de notificaciones 35
5.22 GEN1 – Registro y consulta de bitácoras 36
5.23 Website Administrativo 37
5. Diagramas de base de datos 38
3.1 Usuarios_DB 38
3.2 Oferta_Academica_DB 39
3.3 Ubicaciones_DB 40
3.4 Notificaciones_DB 41
3.5 Notas_Academicas_DB 42
3.6 Matriculas_DB 43
3.7 Seguridad_Auditoria_DB 44
3.8 Facturacion_DB 44
3.9 Configuraciones_DB 45
6. Conclusiones y recomendaciones 46
6.1 Conclusiones 46
6.2 Recomendaciones 46
7. Bibliografía 48

1. Introducción

El Colegio Universitario de Cartago (CUC) tiene una necesidad general de software que apoye la gestión de sus distintos departamentos y que facilite las actividades diarias de colaboradores, estudiantes y profesores, de modo que los servicios brindados al estudiantado sean fácilmente disponibles y accesibles. Para atender esa necesidad se propone el Proyecto Nuevo Avatar, desarrollado durante el curso de Programación V.

MetaU es la propuesta de nuestro equipo para reemplazar la solución actual, Avatar. Se trata de una plataforma que centraliza y automatiza los procesos más importantes de la institución: la gestión de usuarios y roles, la oferta académica, la matrícula, el control de notas, la facturación y las notificaciones. Su construcción se basa en una arquitectura de microservicios que exponen servicios REST, lo que permite que cada componente sea independiente, escalable y mantenible, con autenticación por tokens y trazabilidad de las acciones a través de bitácoras.

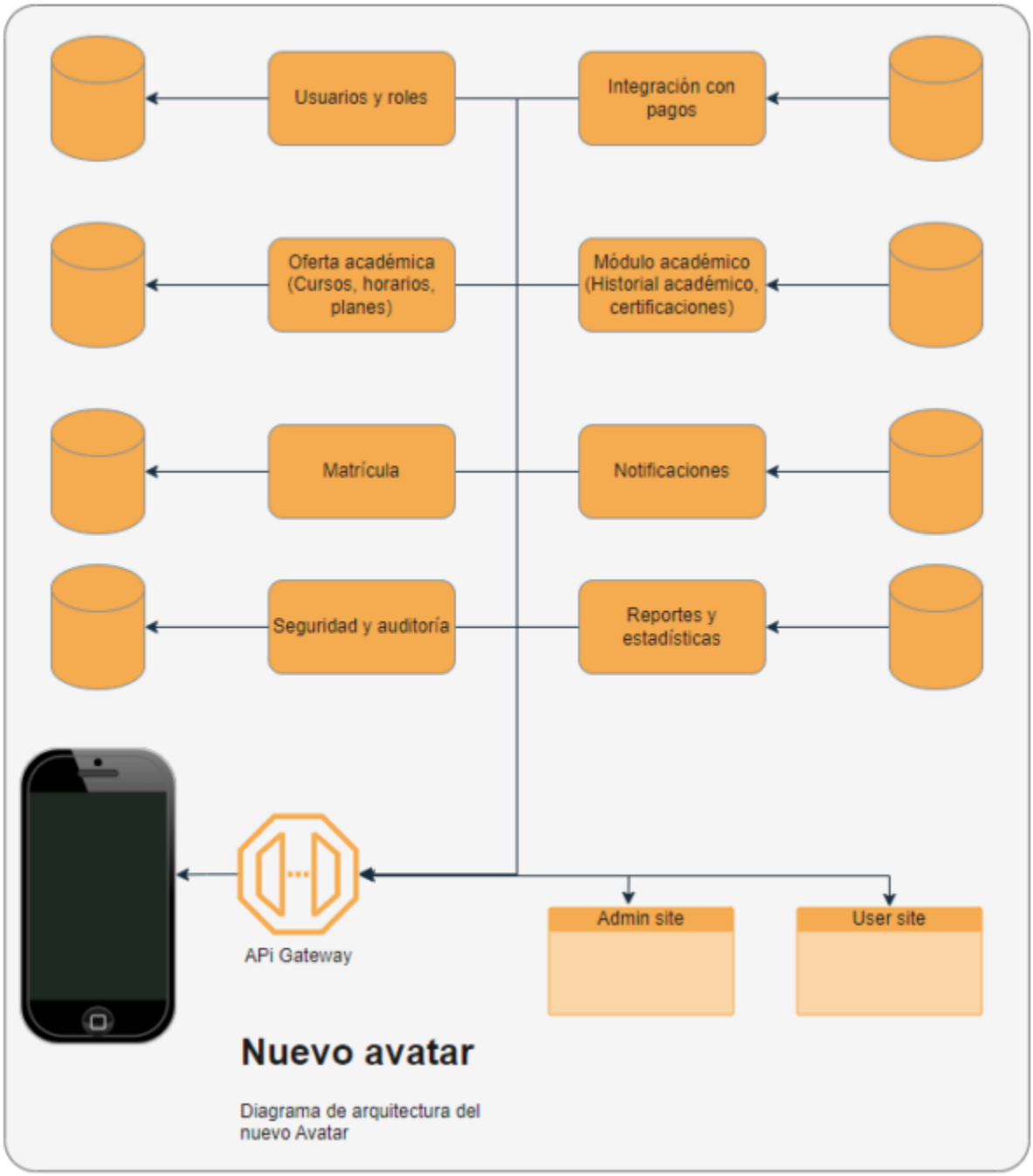
El alcance de este primer sprint comprende 22 historias de usuario, organizadas en los siguientes módulos:

- **Usuarios y roles:** administración de usuarios, roles, parámetros y módulos, además del servicio de login.
- **Oferta académica:** instituciones, carreras, cursos, grupos, periodos y profesores.
- **Matrícula:** prematrícula, matrícula, expedientes de estudiantes, direcciones y notas.
- **Académico:** historial académico y listado de estudiantes matriculados por periodo.
- **Integración de pagos y notificaciones:** facturación, pagos y envío de notificaciones por correo electrónico.
- **General:** registro y consulta de bitácoras de todas las acciones del sistema.

Para su desarrollo, el equipo trabaja bajo el modelo Scrum y utiliza .NET 10 con ASP.NET Core Web API (Minimal APIs) para el backend, SQL Server como base de datos alojada en Plesk, y Git y GitHub para el control de versiones, siguiendo una estructura de ramas Main, Develop y Feature con cambios integrados mediante Pull Requests.

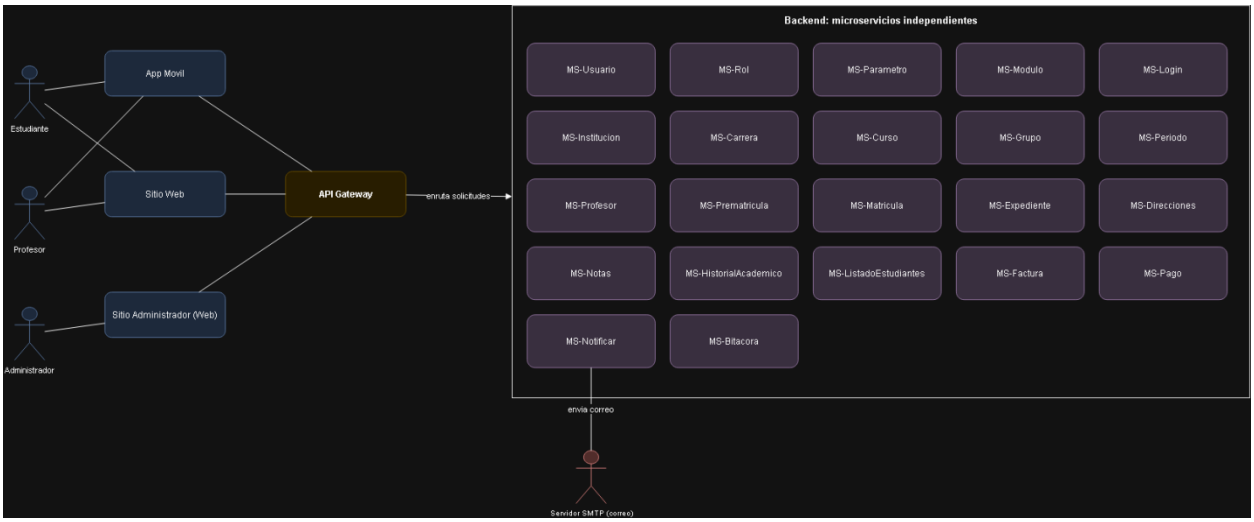
El presente documento reúne el análisis y diseño de la solución: la arquitectura general, los diagramas de base de datos, los casos de uso y los diagramas de clases de los servicios, así como las conclusiones y recomendaciones del equipo. La evidencia de las pruebas técnicas se presenta en un documento independiente dentro del repositorio oficial del proyecto.

2. Arquitectura general del sistema

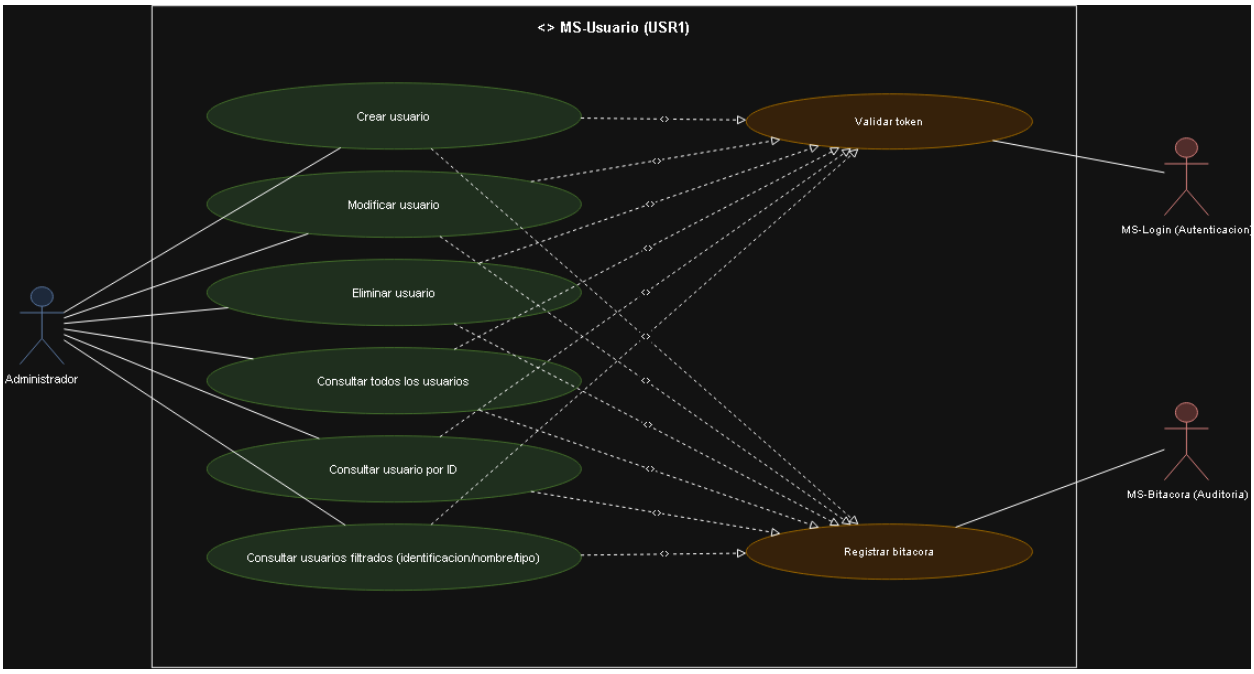


4. Diagramas de casos de uso

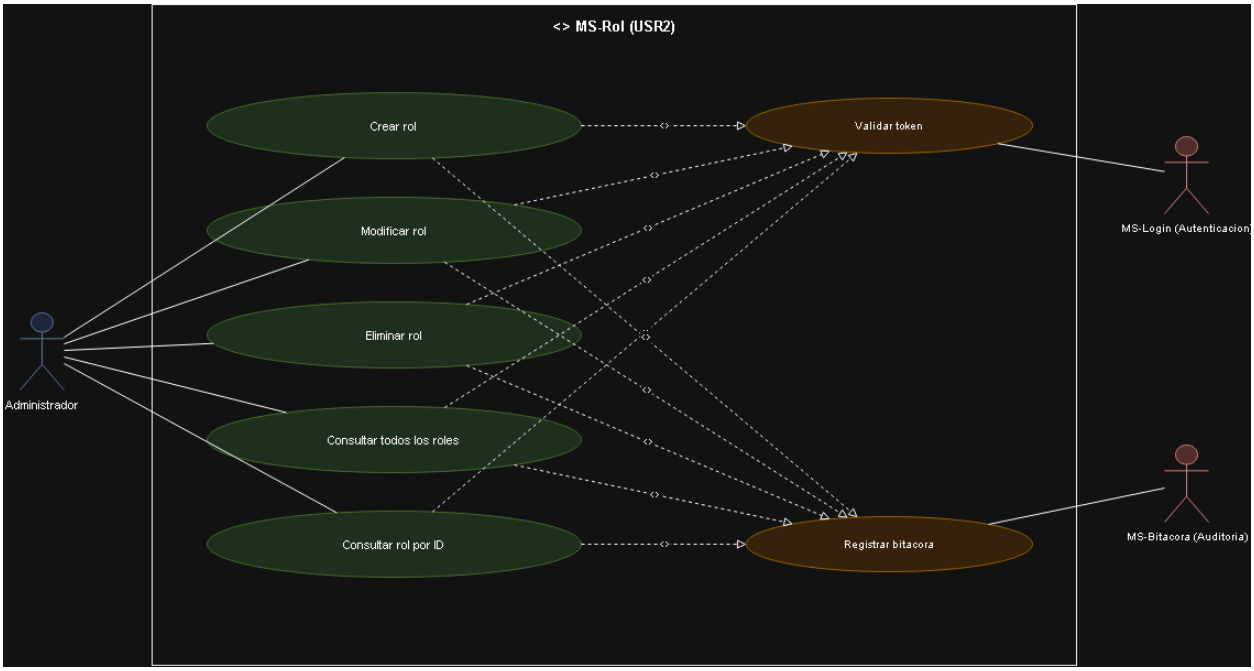
4.1 Caso de Uso de la Arquitectura



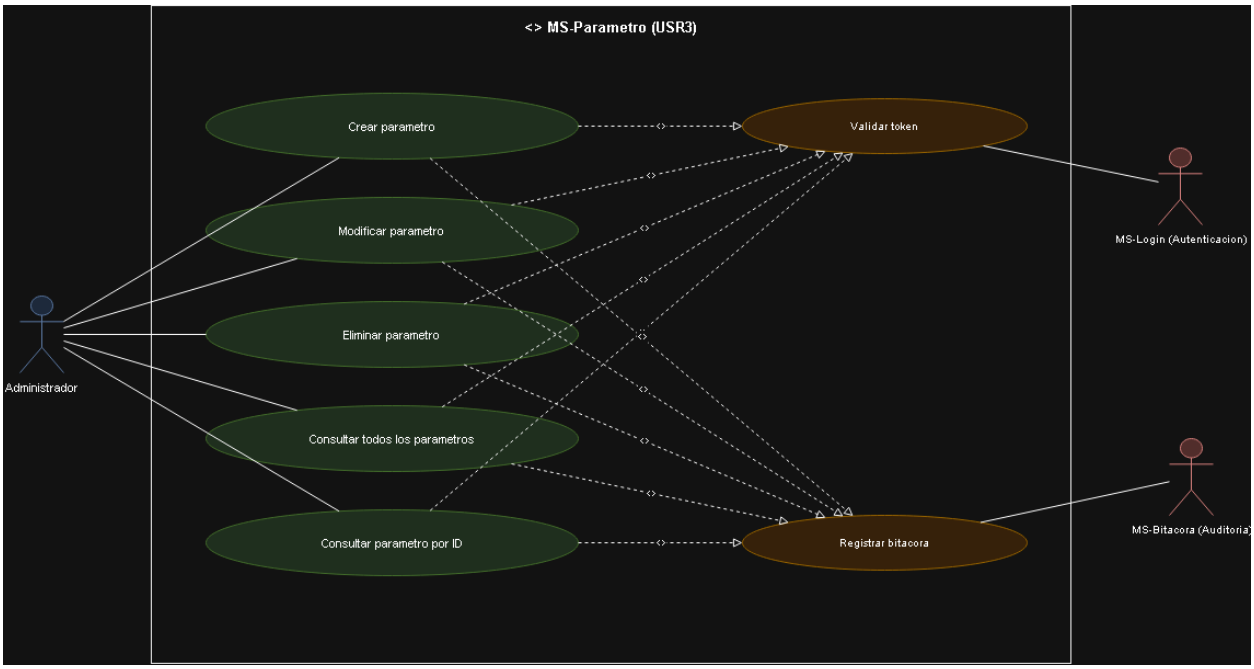
4.2 Caso de Uso del Microservicio Usuarios



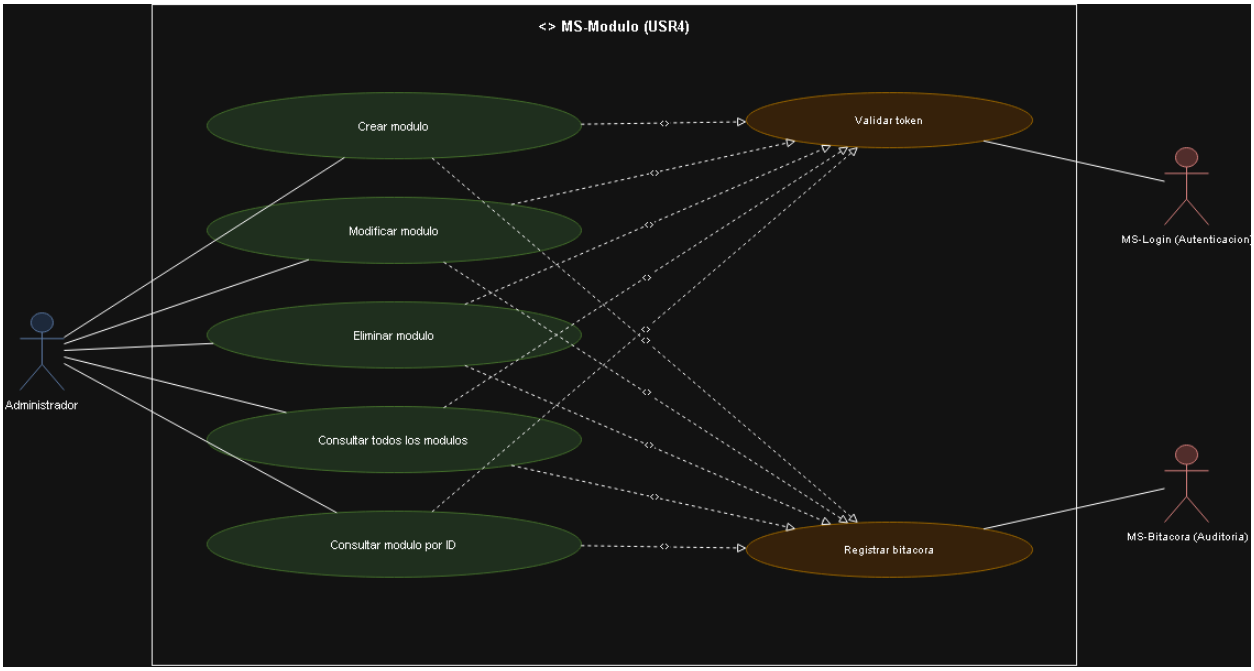
4.3 Caso de Uso del Microservicio Roles



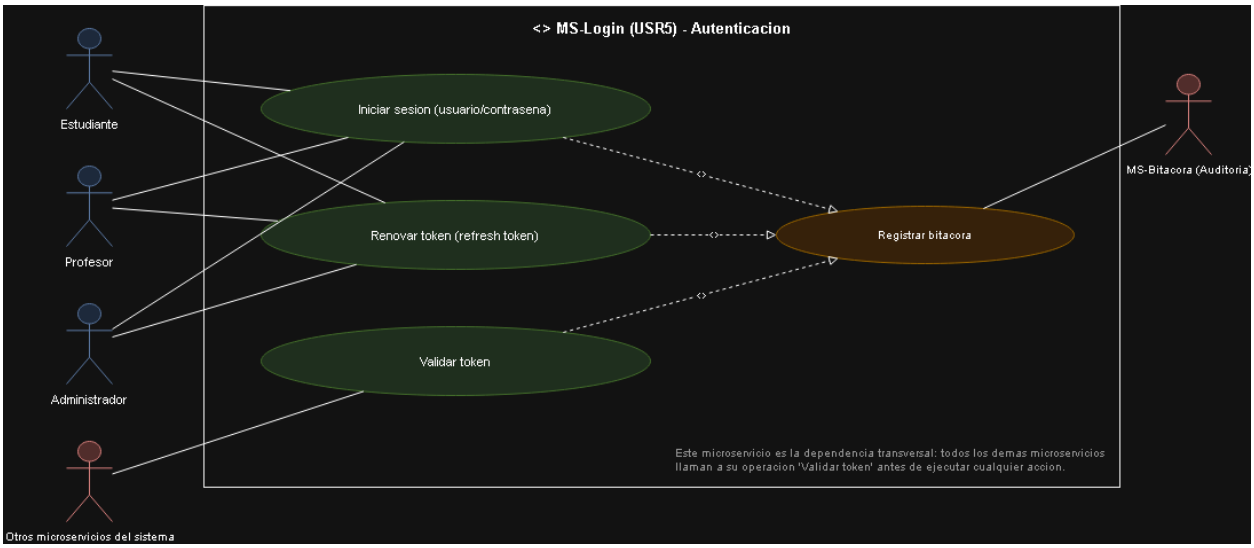
4.4 Caso de Uso del Microservicio Parámetros



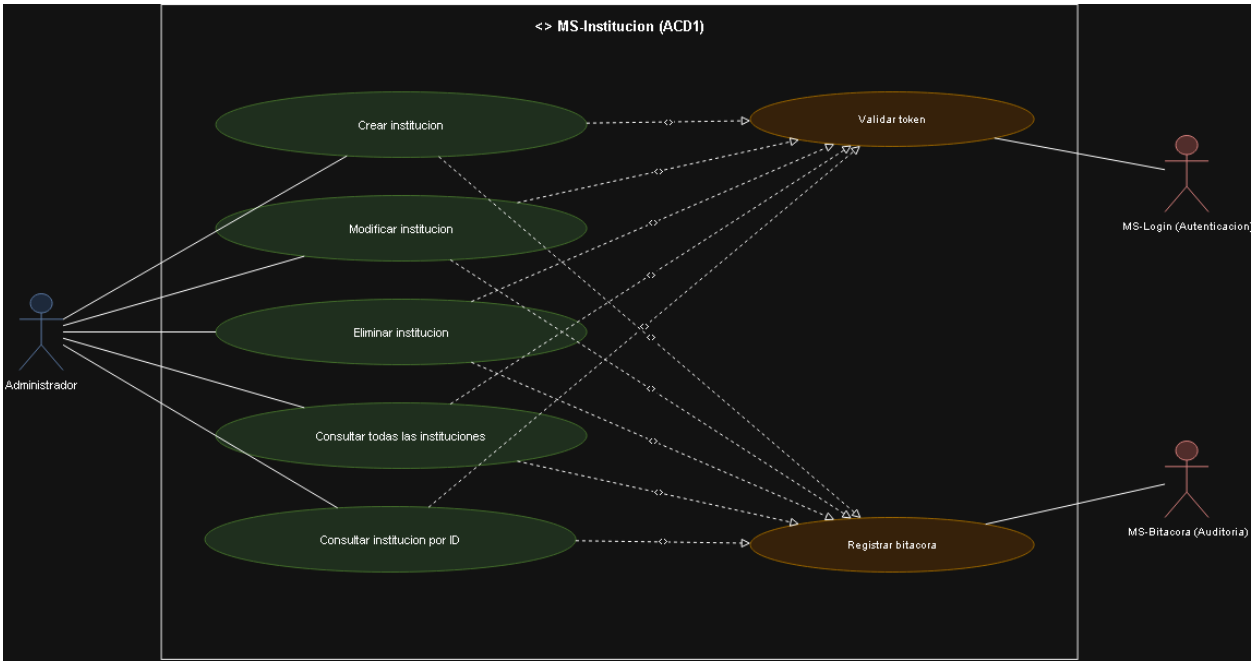
4.5 Caso de Uso del Microservicio Módulos



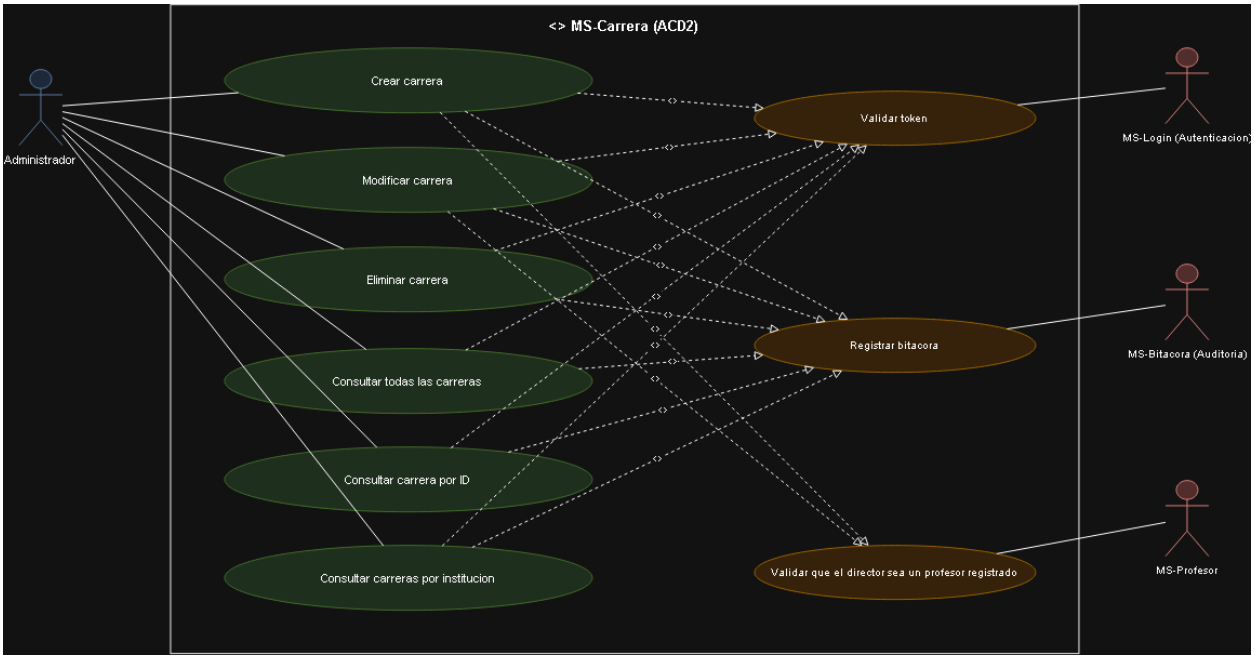
4.6 Caso de Uso del Microservicio Login



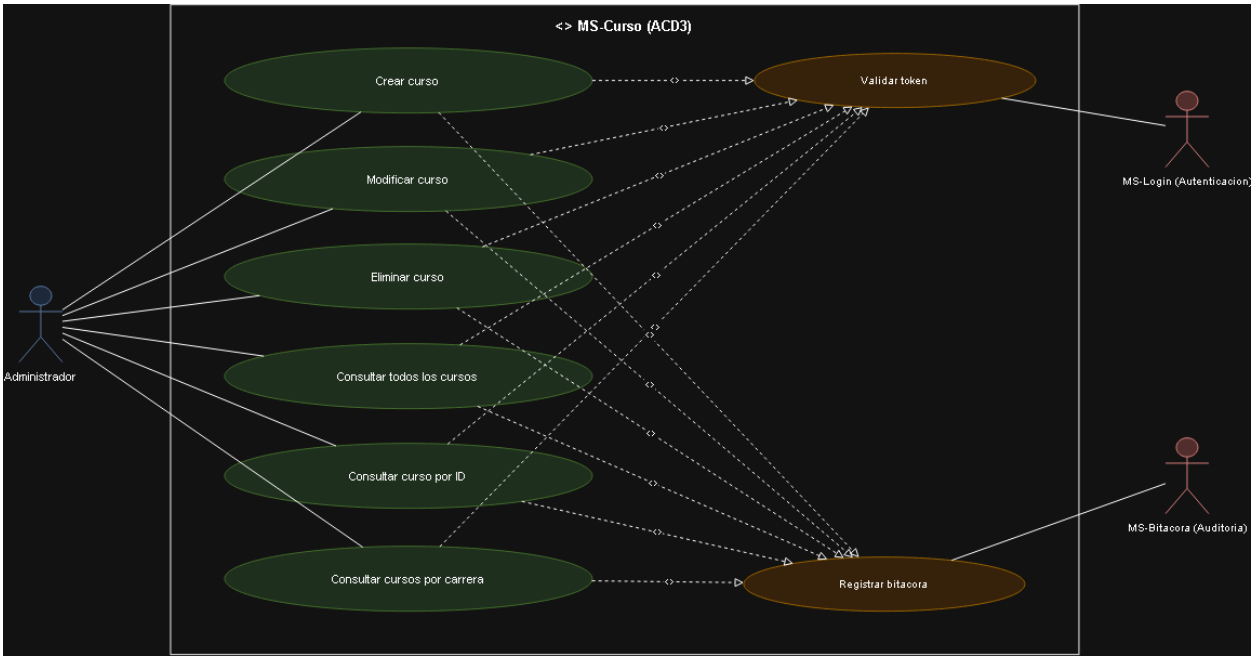
4.7 Caso de Uso del Microservicio Instituciones



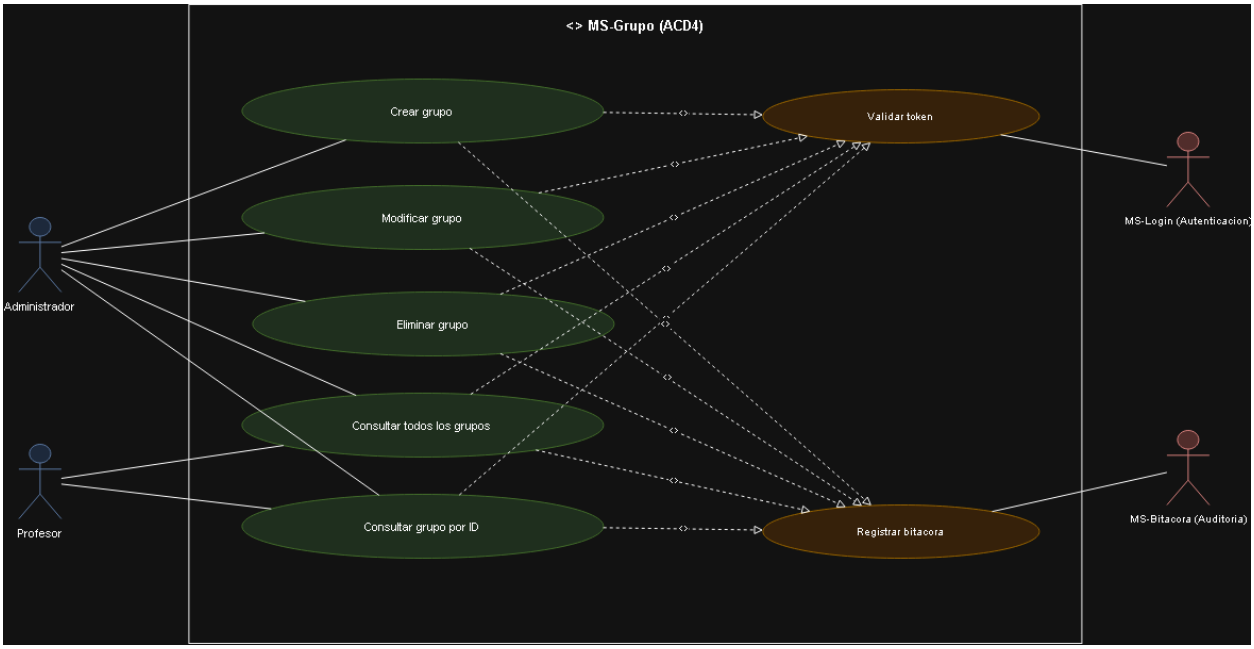
4.7 Caso de Uso del Microservicio Carreras



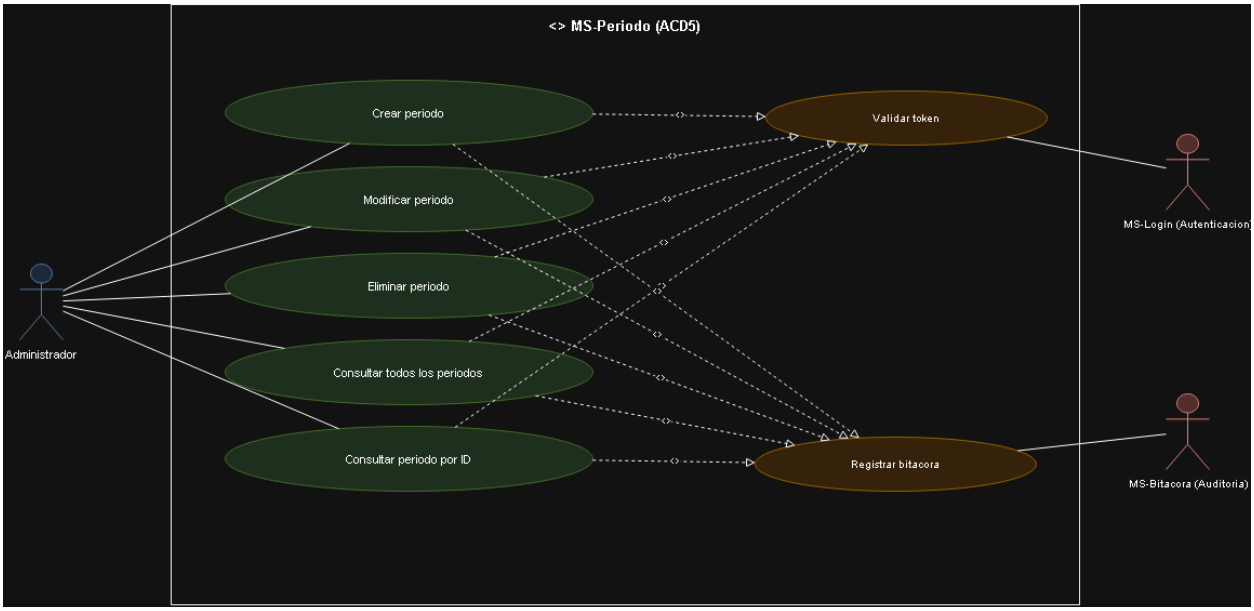
4.8 Caso de Uso del Microservicio Cursos



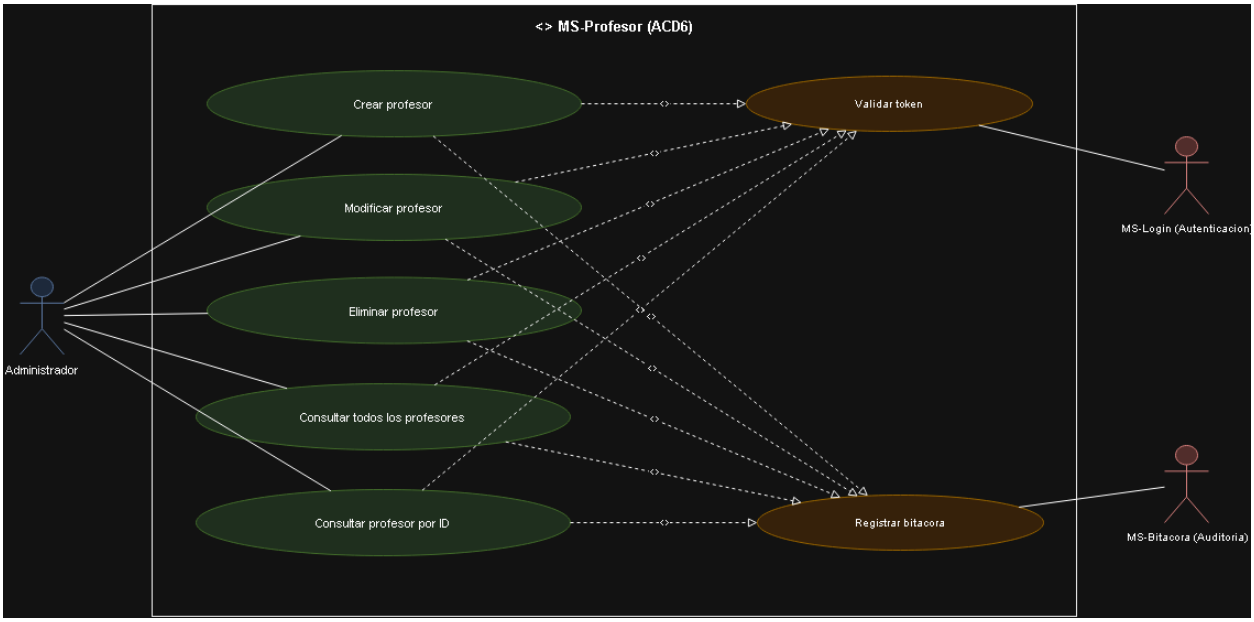
4.9 Caso de Uso del Microservicio Grupos



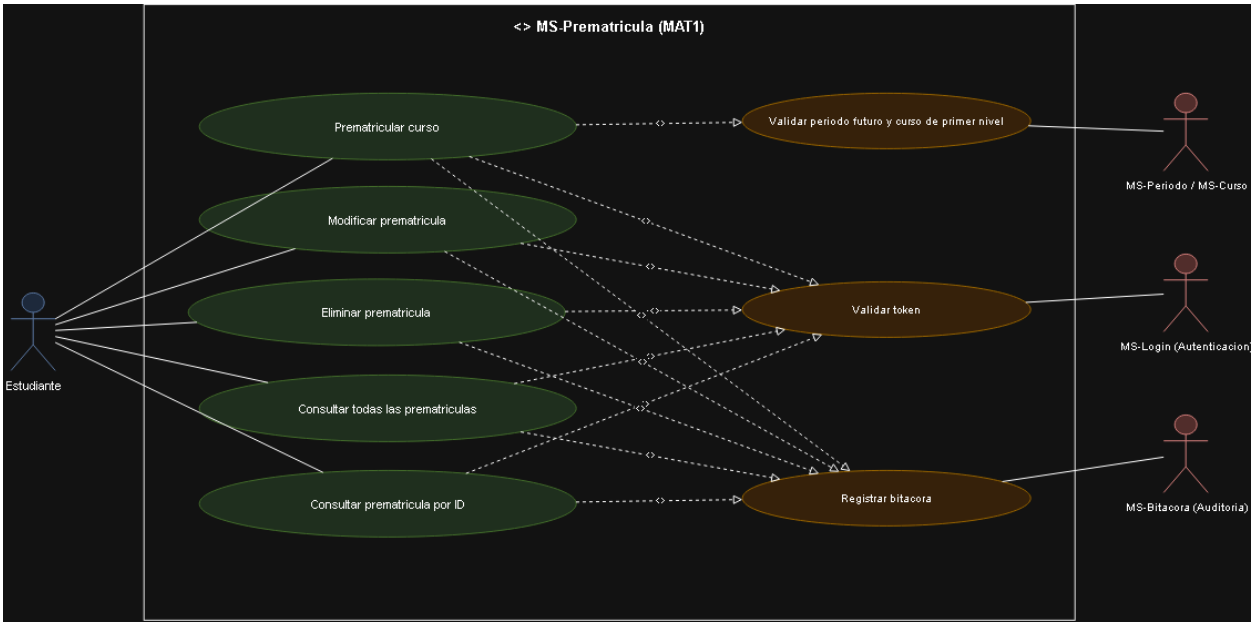
4.10 Caso de Uso del Microservicio Periodos



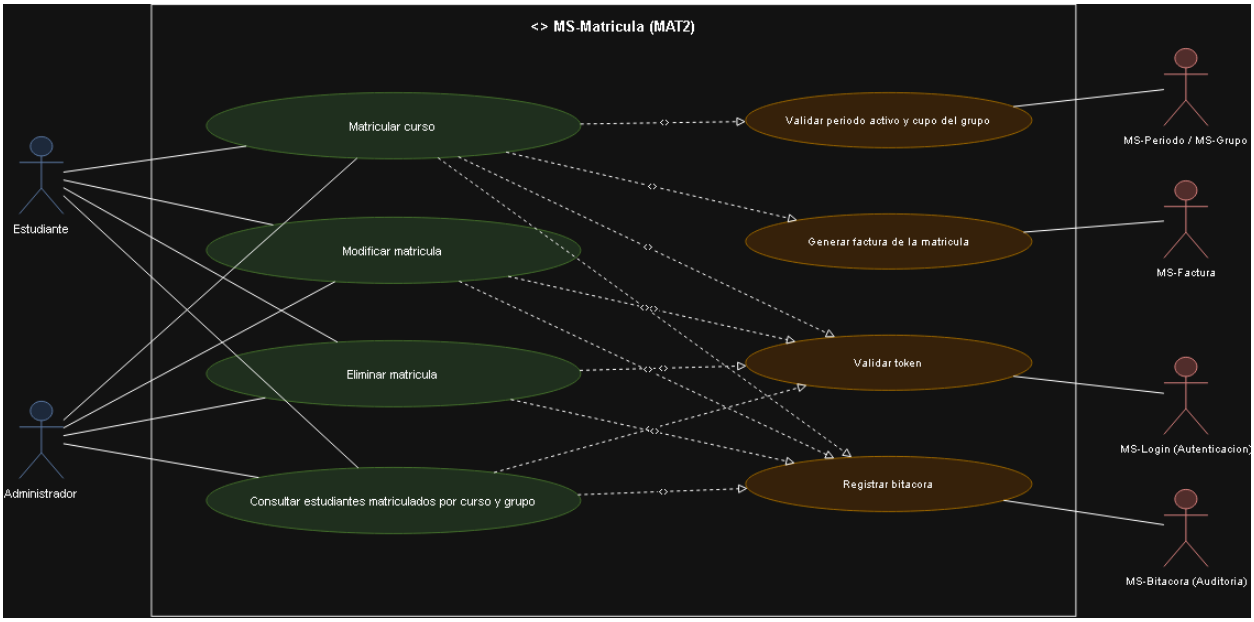
4.11 Caso de Uso del Microservicio Profesores



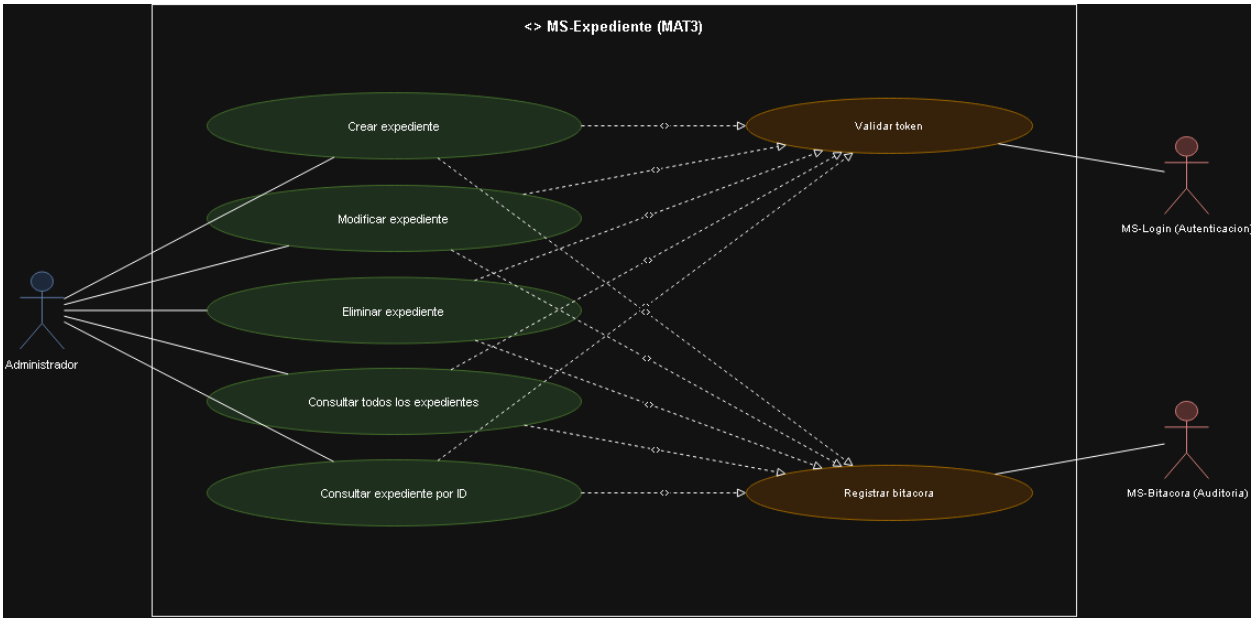
4.12 Caso de Uso del Microservicio Pre-Matriculas



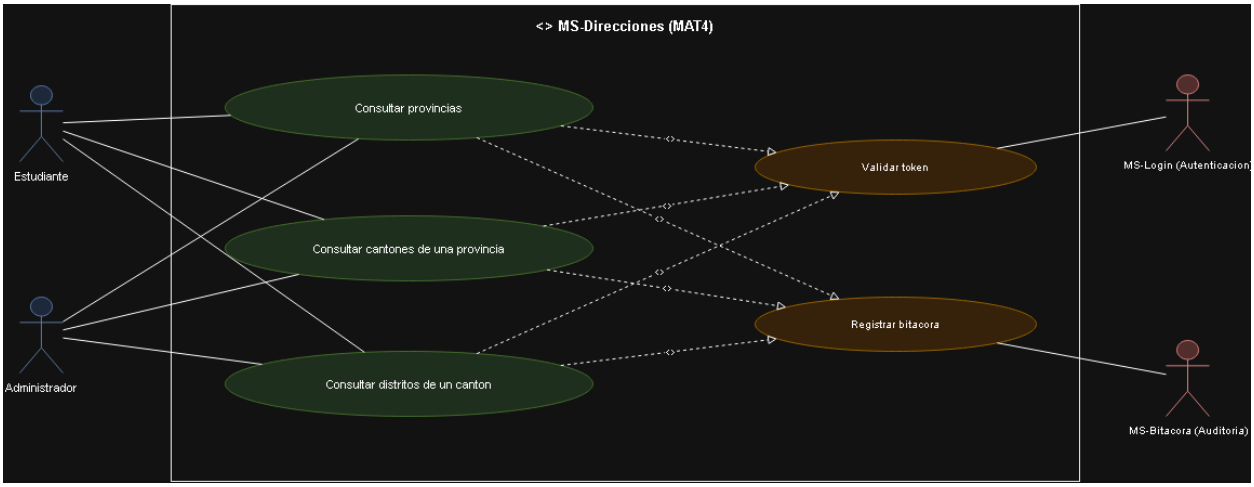
4.13 Caso de Uso del Microservicio Matriculas



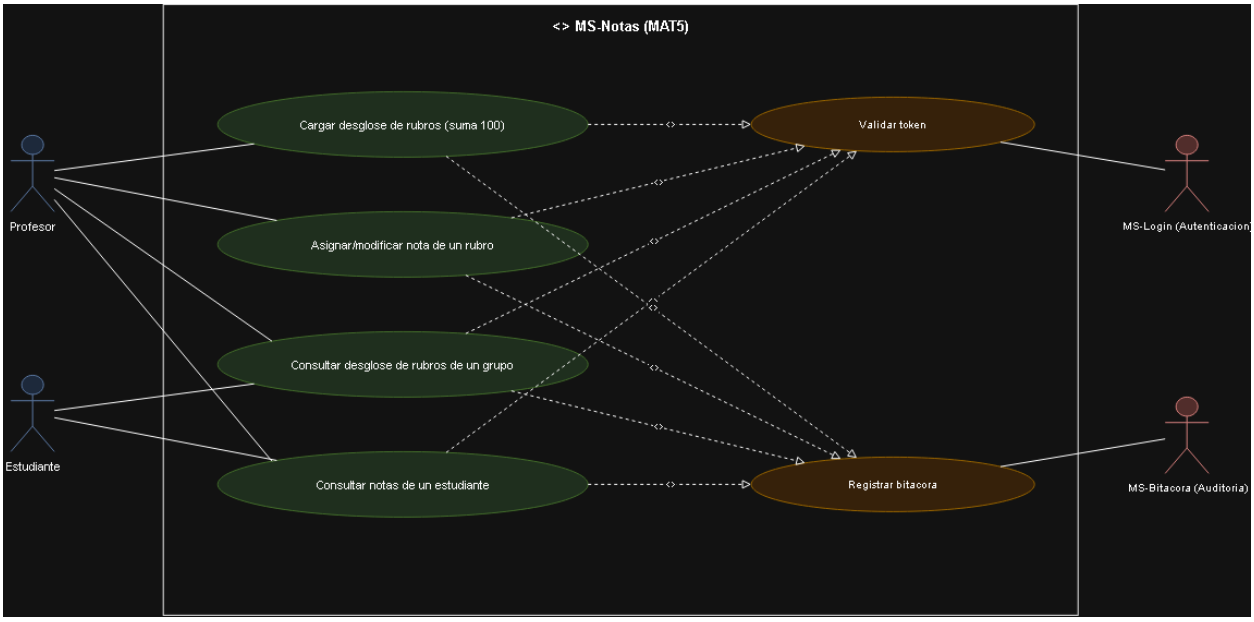
4.14 Caso de Uso del Microservicio Expediente



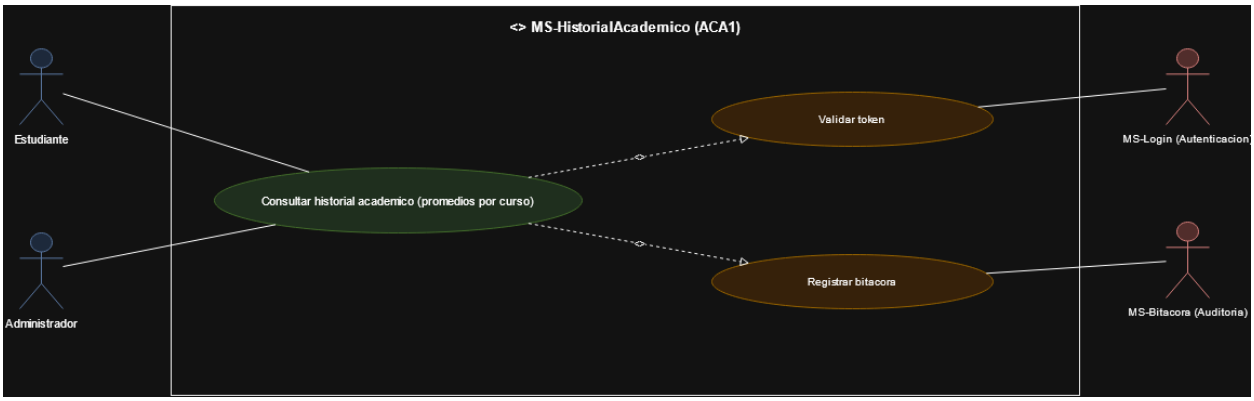
4.15 Caso de Uso del Microservicio Direcciones



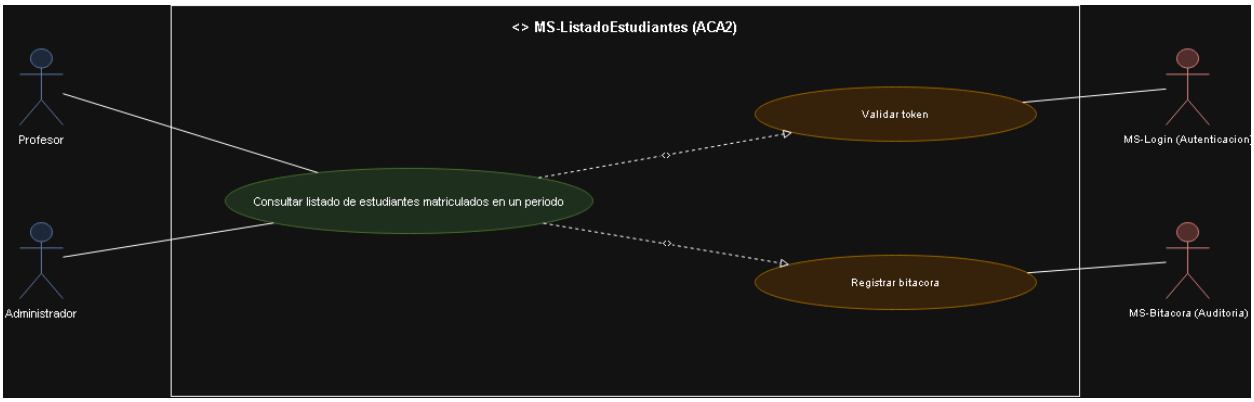
4.16 Caso de Uso del Microservicio Notas



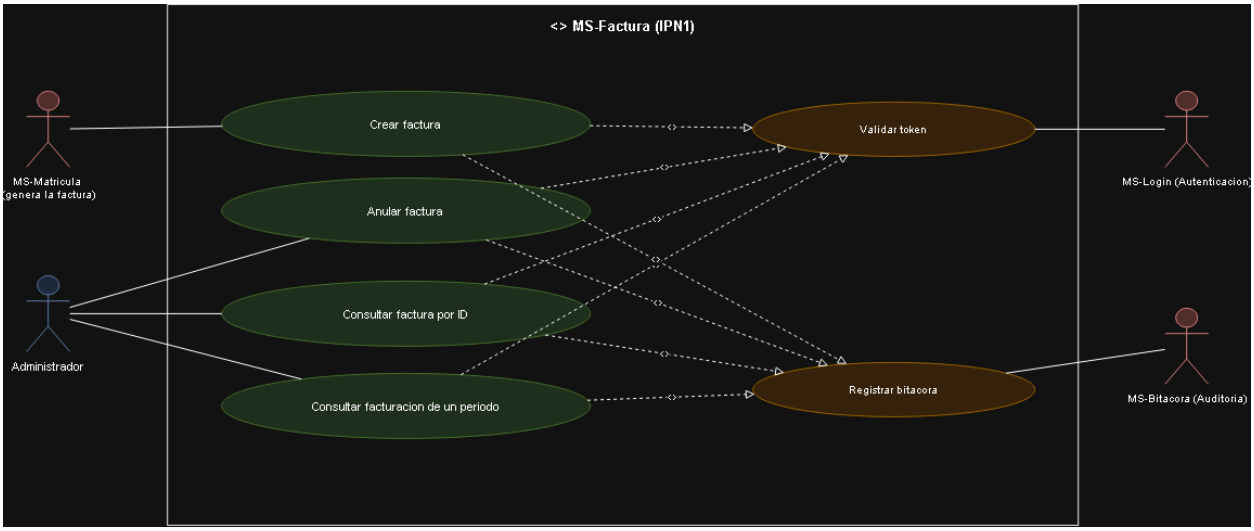
4.17 Caso de Uso del Microservicio Historial Academico



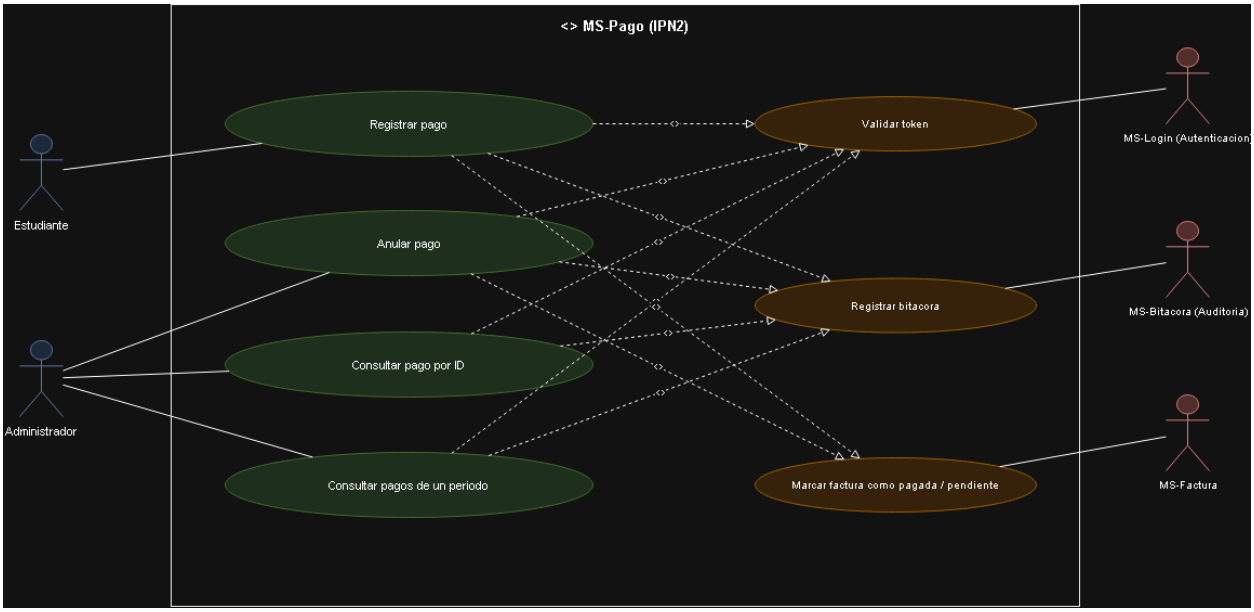
4.18 Caso de Uso del Microservicio Listado Estudiantes



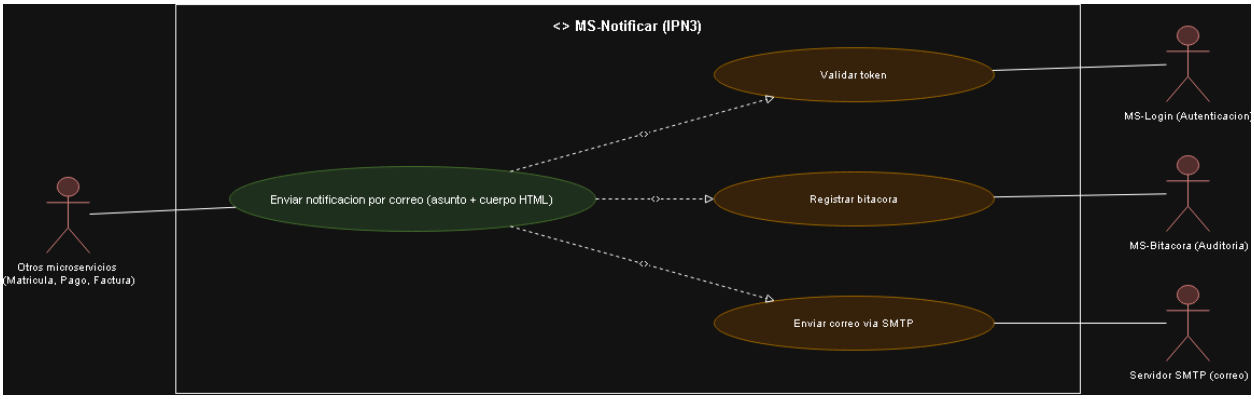
4.19 Caso de Uso del Microservicio Facturas



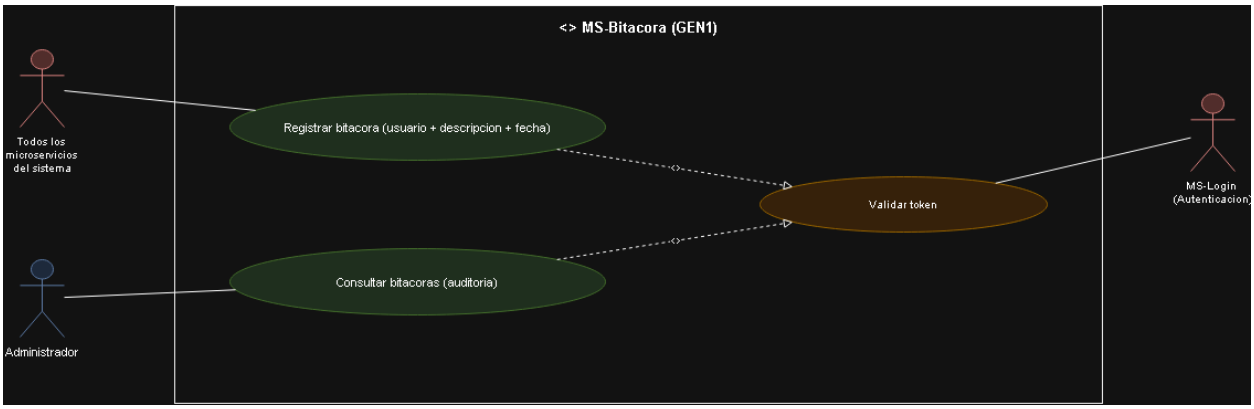
4.20 Caso de Uso del Microservicio Pagos



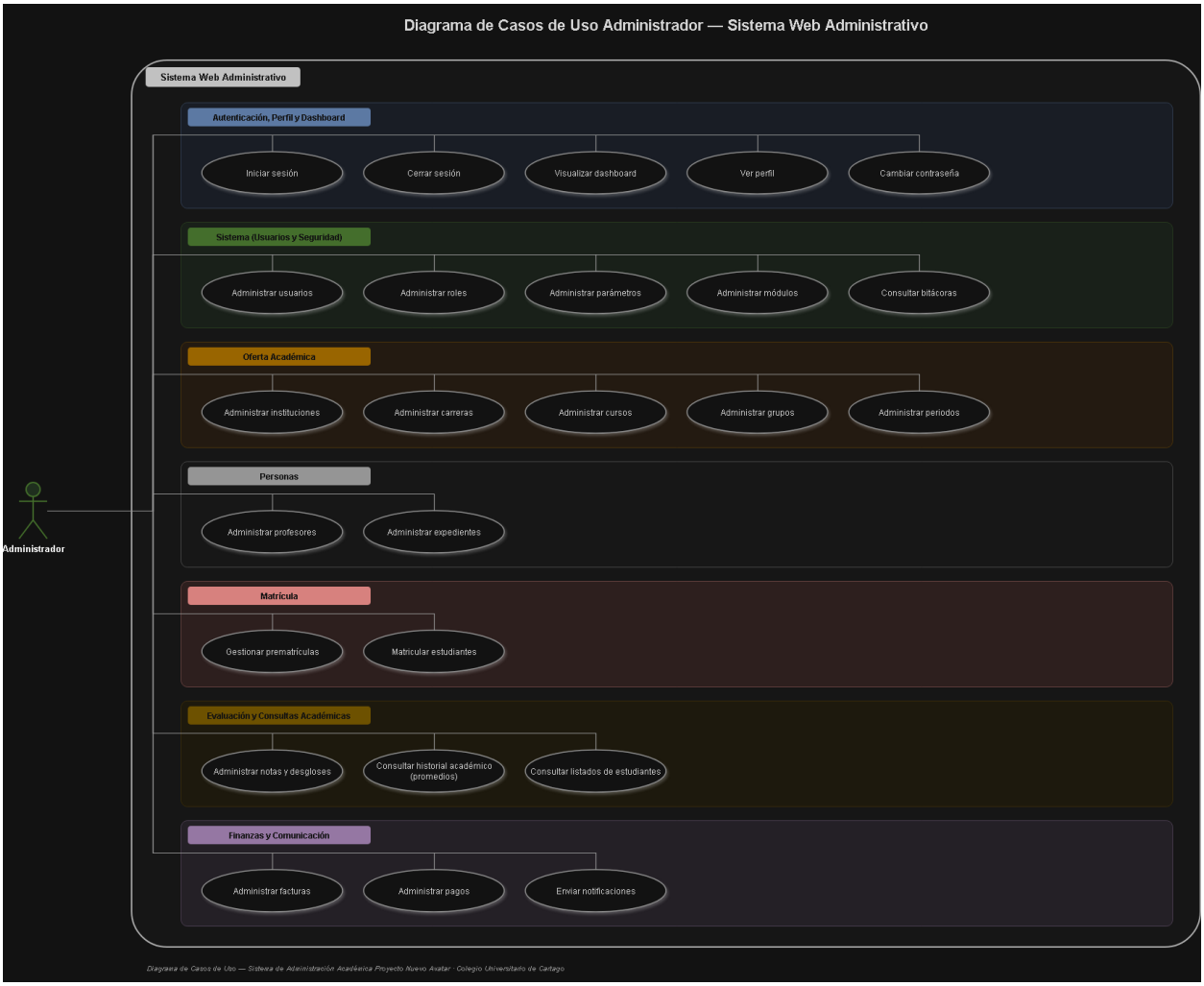
4.21 Caso de Uso del Microservicio Notificaciones



4.22 Caso de Uso del Microservicio Bitácoras

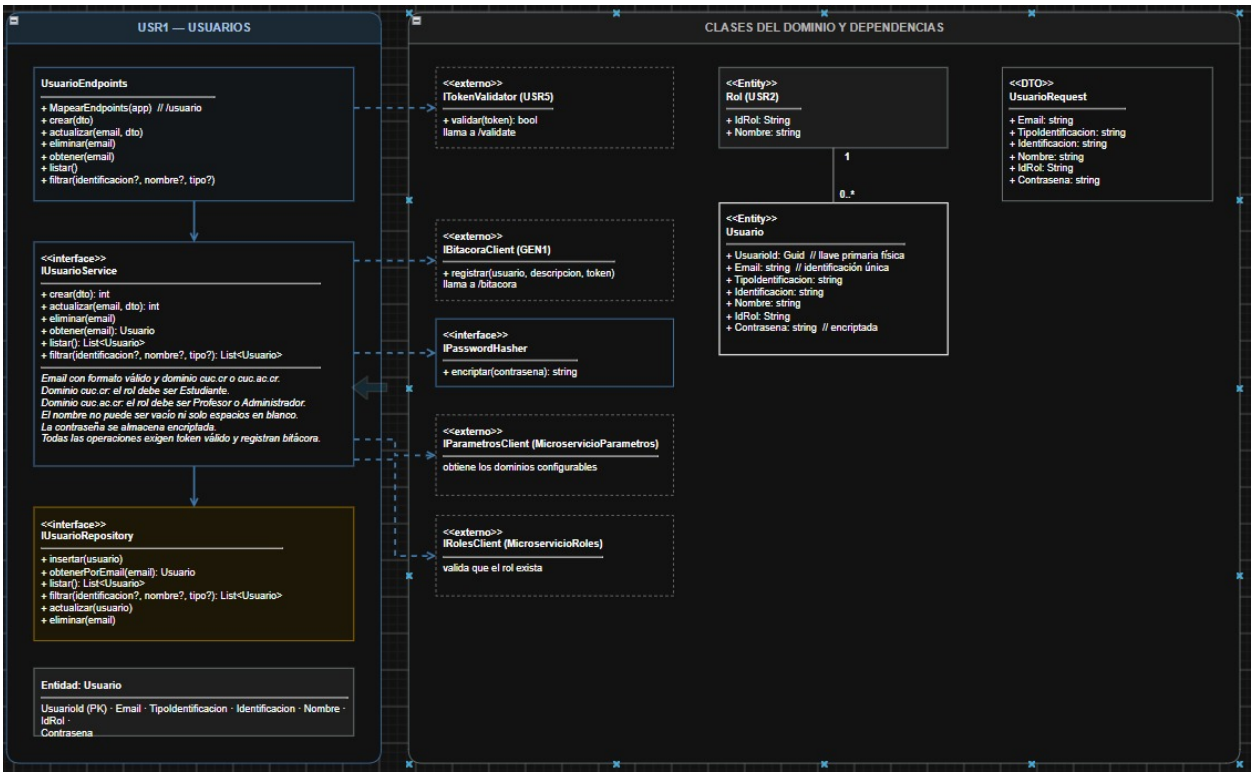


4.23 Caso de Uso General del Website Administrativo

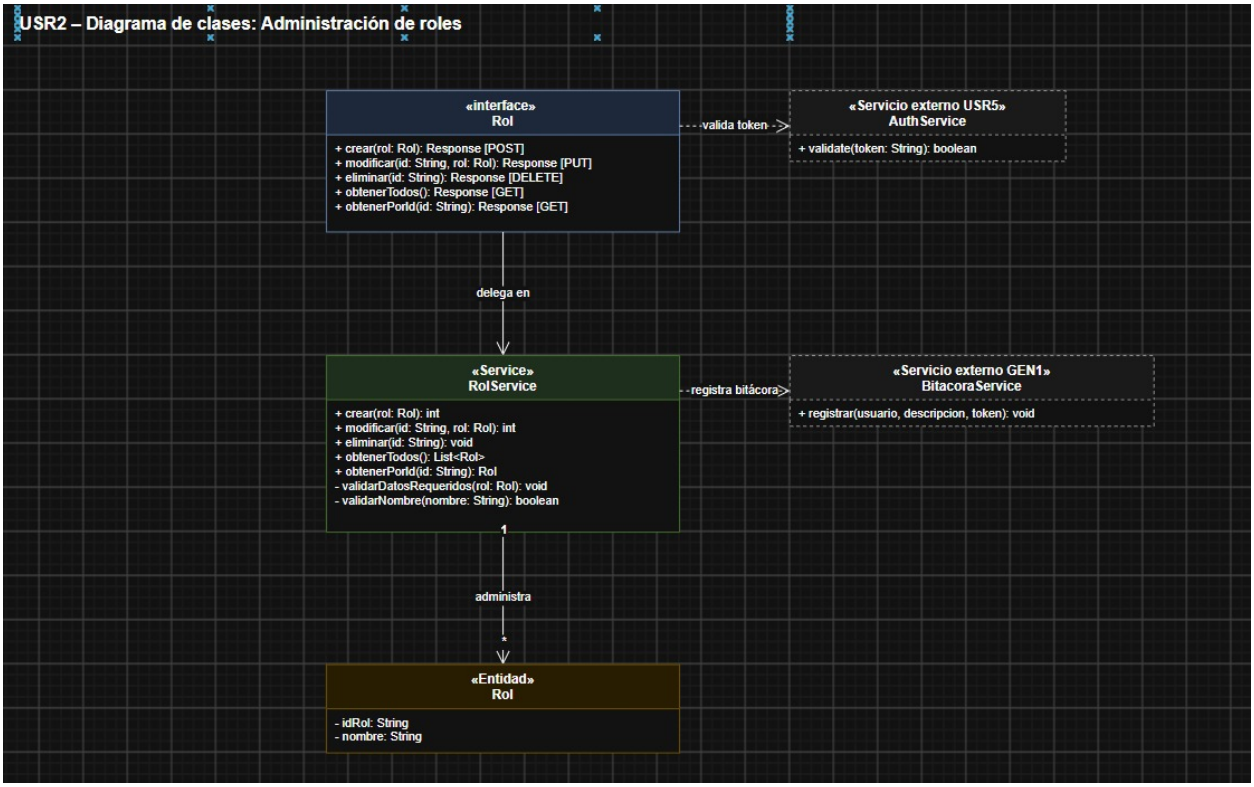


5. Diagramas de clases

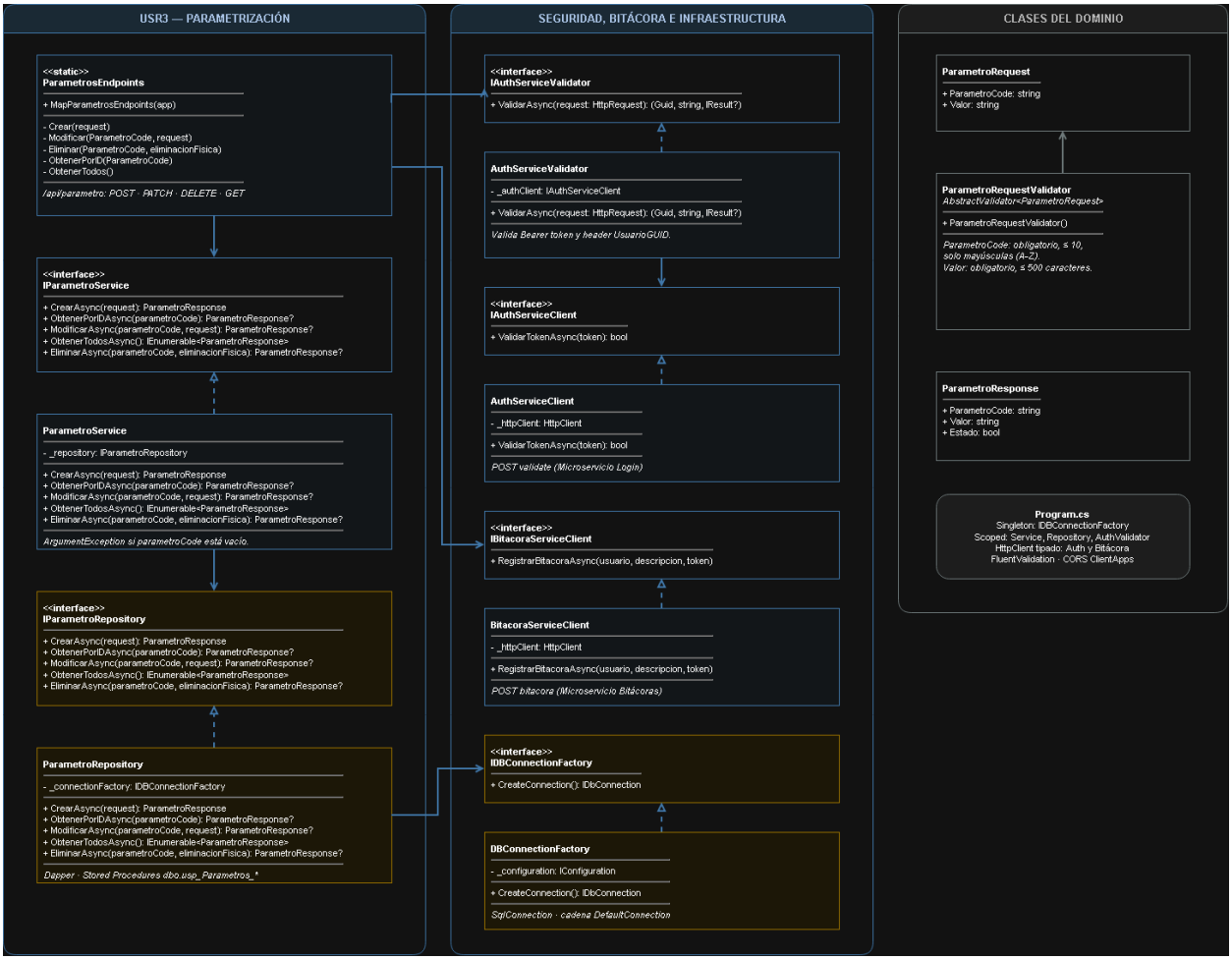
5.1 USR1 – Administración de usuarios



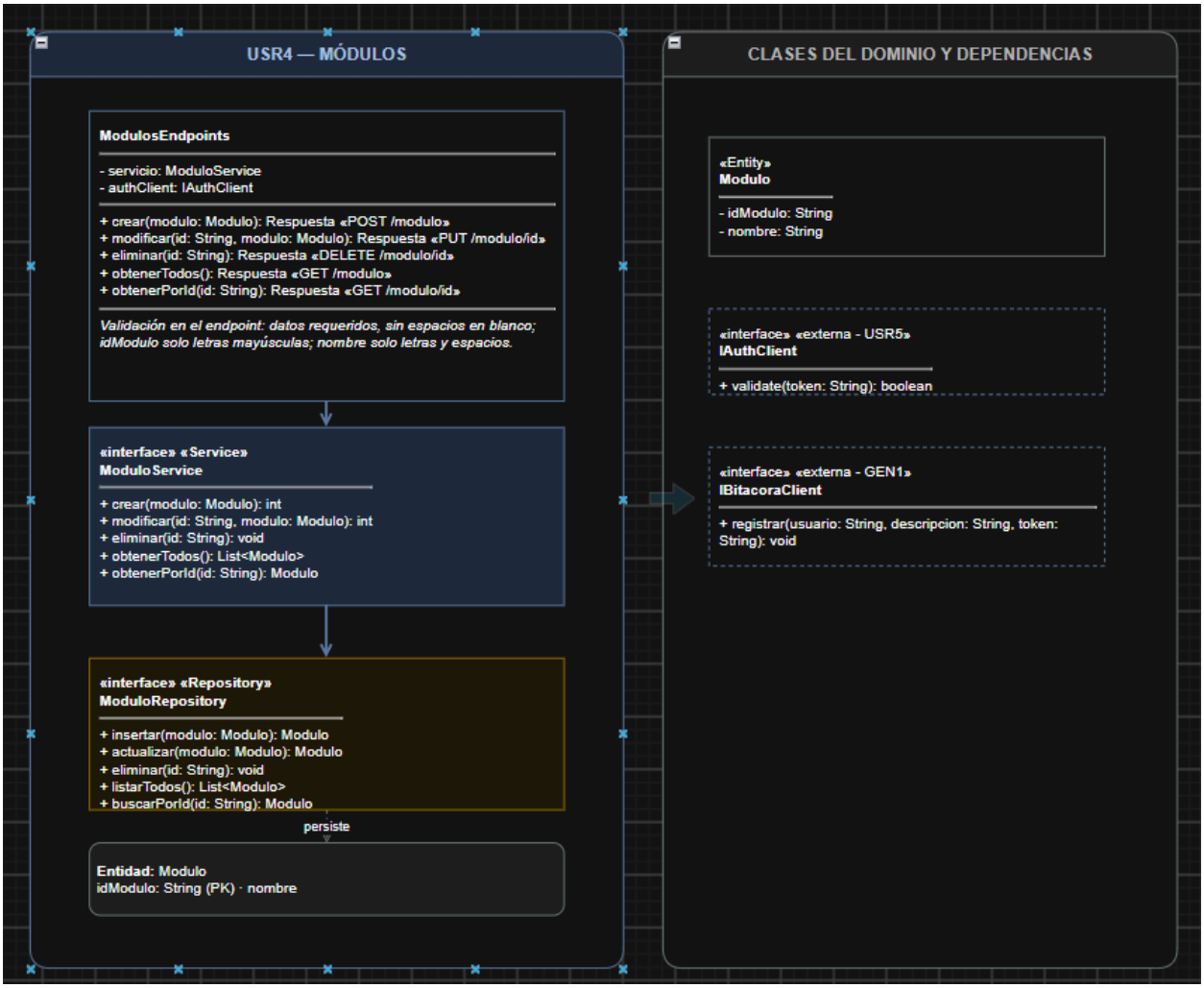
5.2 USR2 – Administración de roles



5.3 USR3 – Administración de parámetros

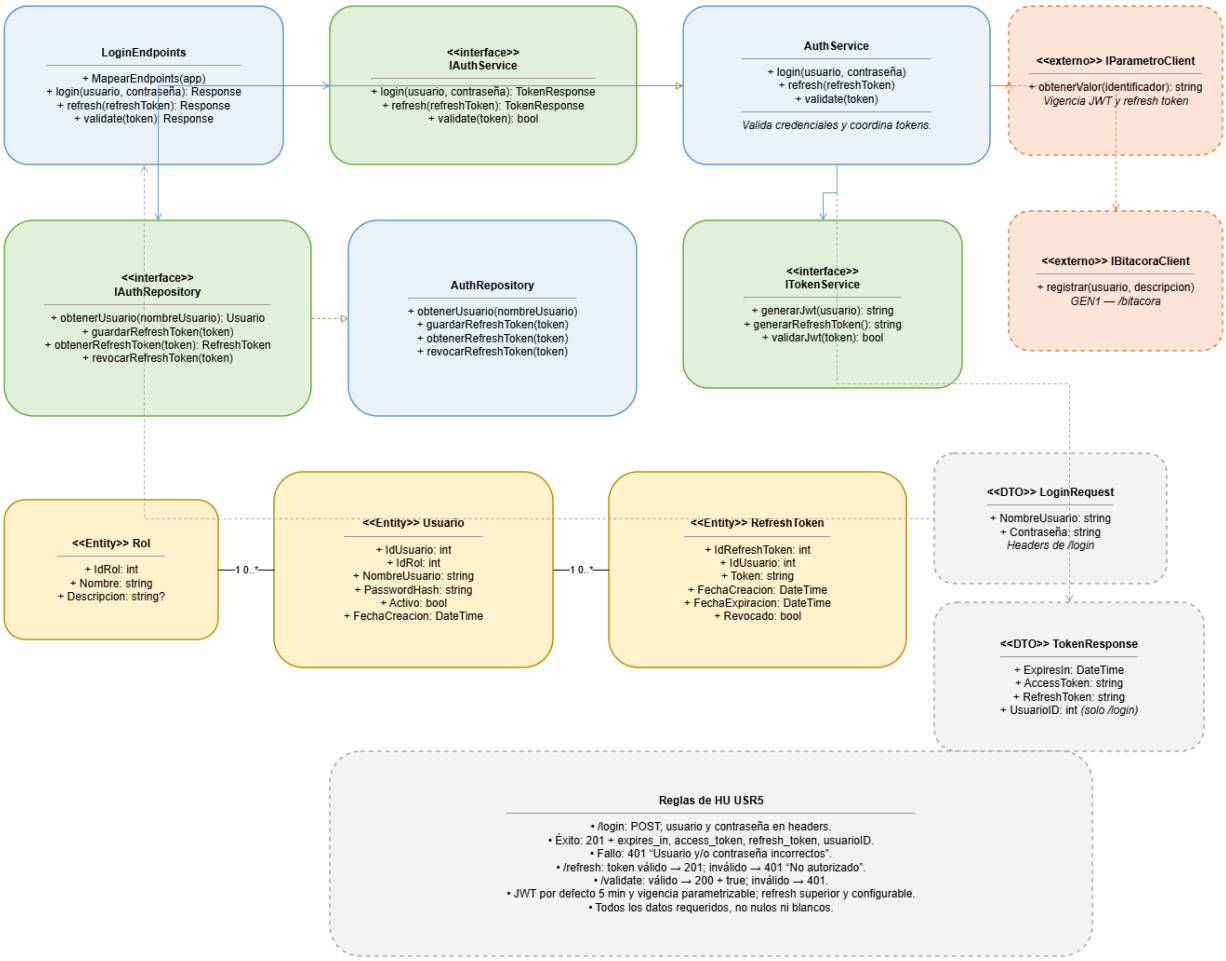


5.4 USR4 – Administración de módulos



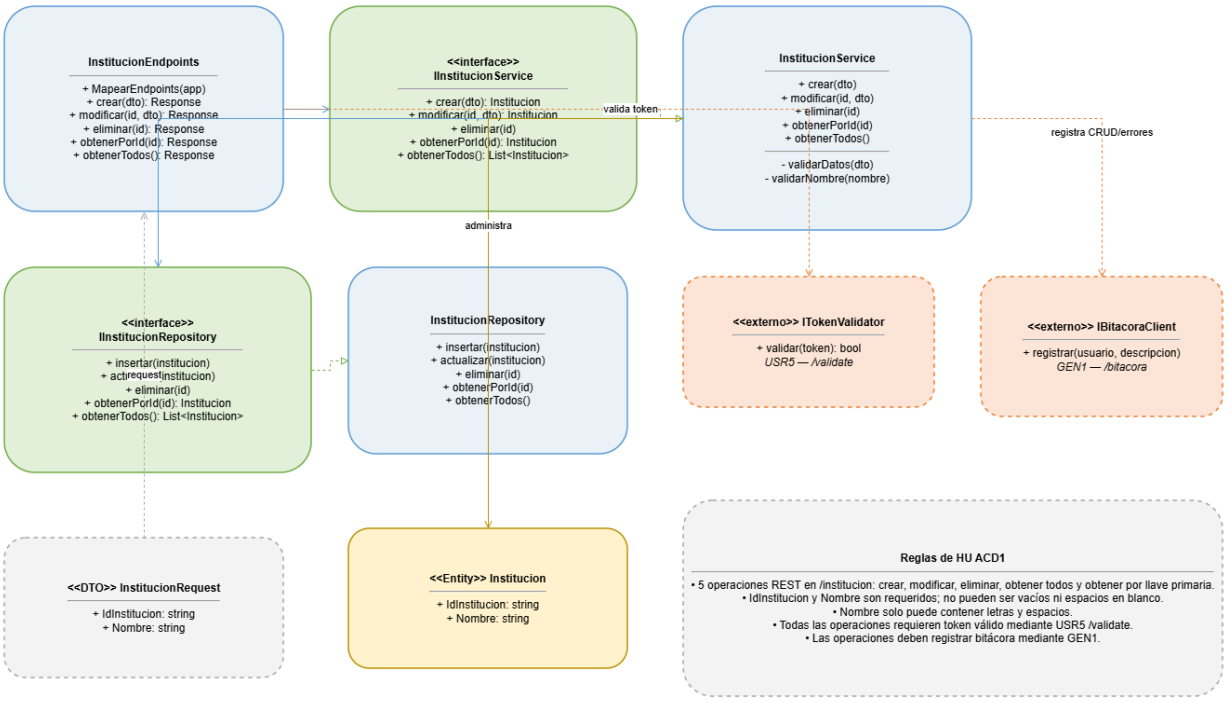
5.5 USR5 – Login y manejo de tokens

USR5 — DIAGRAMA DE CLASES: LOGIN
Autenticación, JWT y refresh token



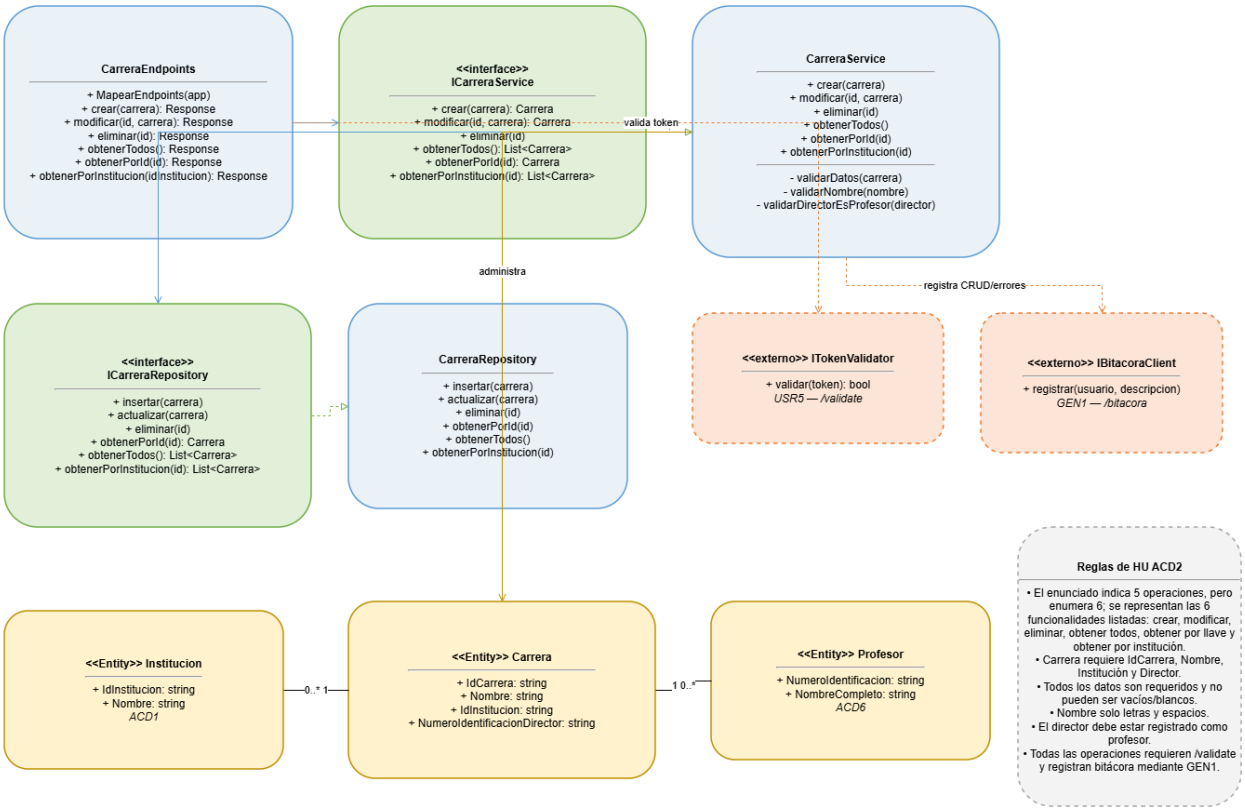
5.6 ACD1 – Administración de instituciones

ACD1 — DIAGRAMA DE CLASES: ADMINISTRACIÓN DE INSTITUCIONES



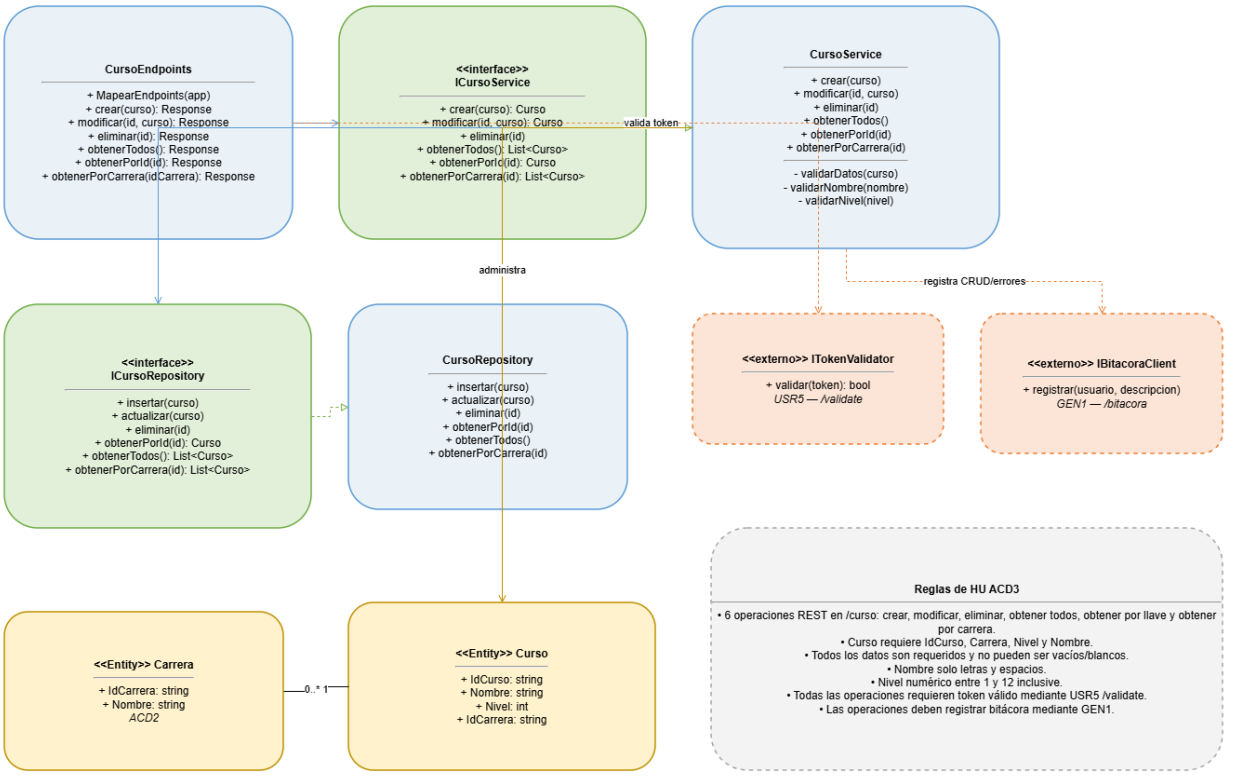
5.7 ACD2 – Administración de carreras

ACD2 — DIAGRAMA DE CLASES: ADMINISTRACIÓN DE CARRERAS

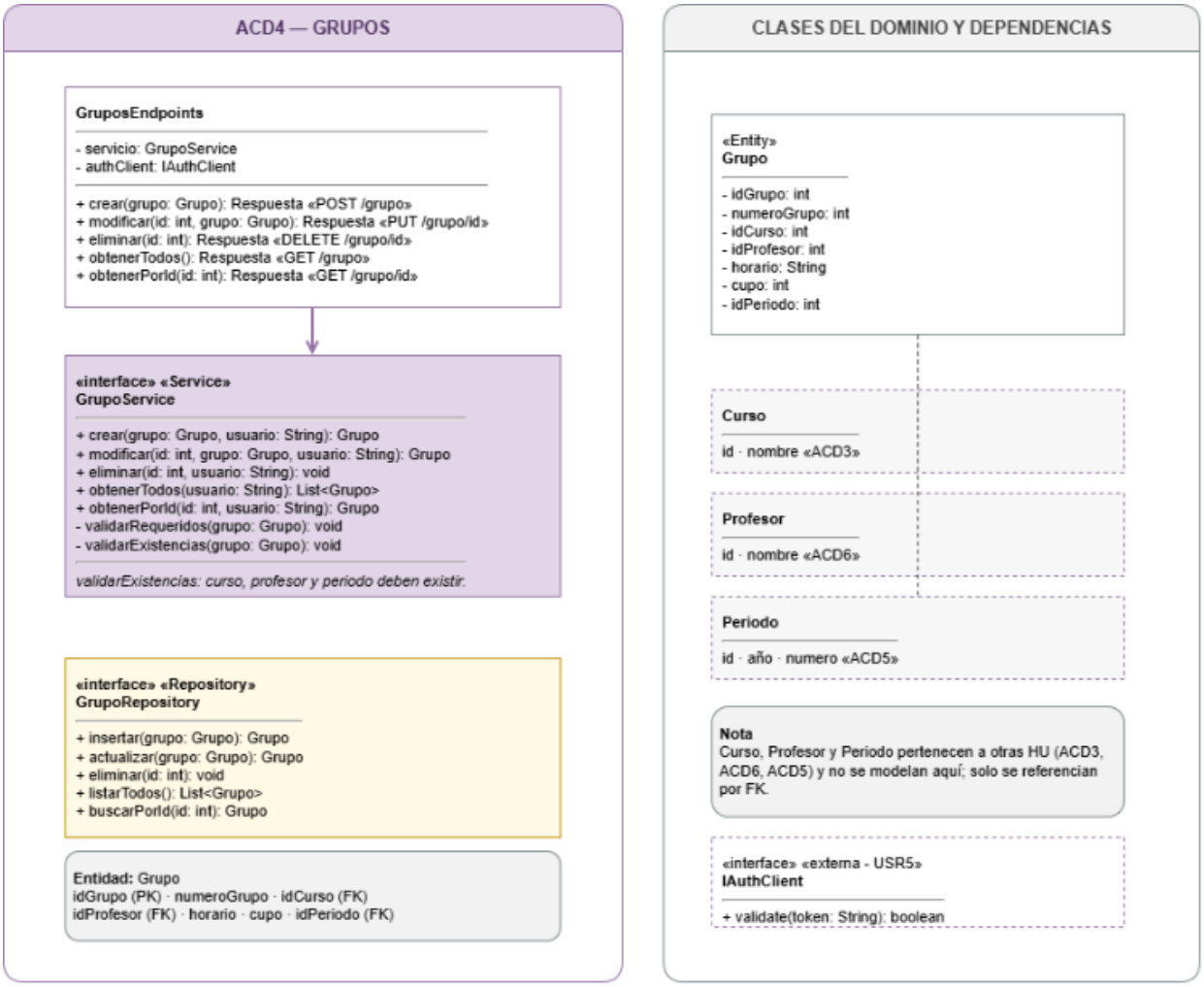


5.8 ACD3 – Administración de cursos

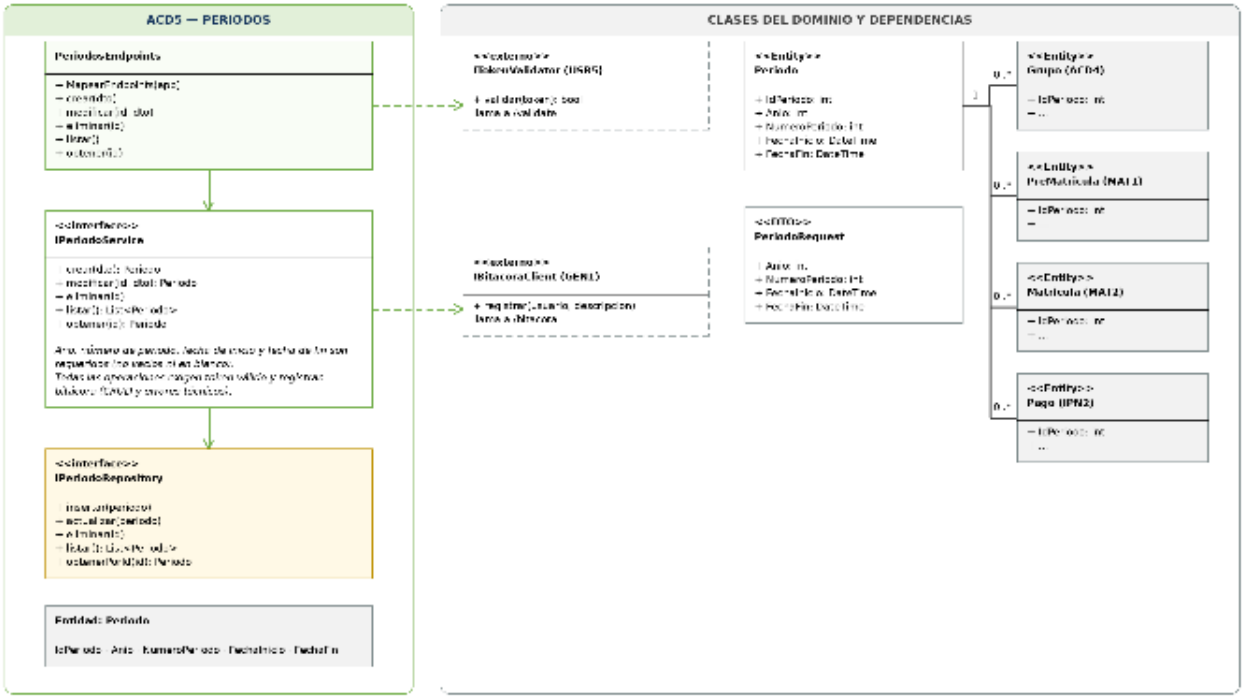
ACD3 — DIAGRAMA DE CLASES: ADMINISTRACIÓN DE CURSOS



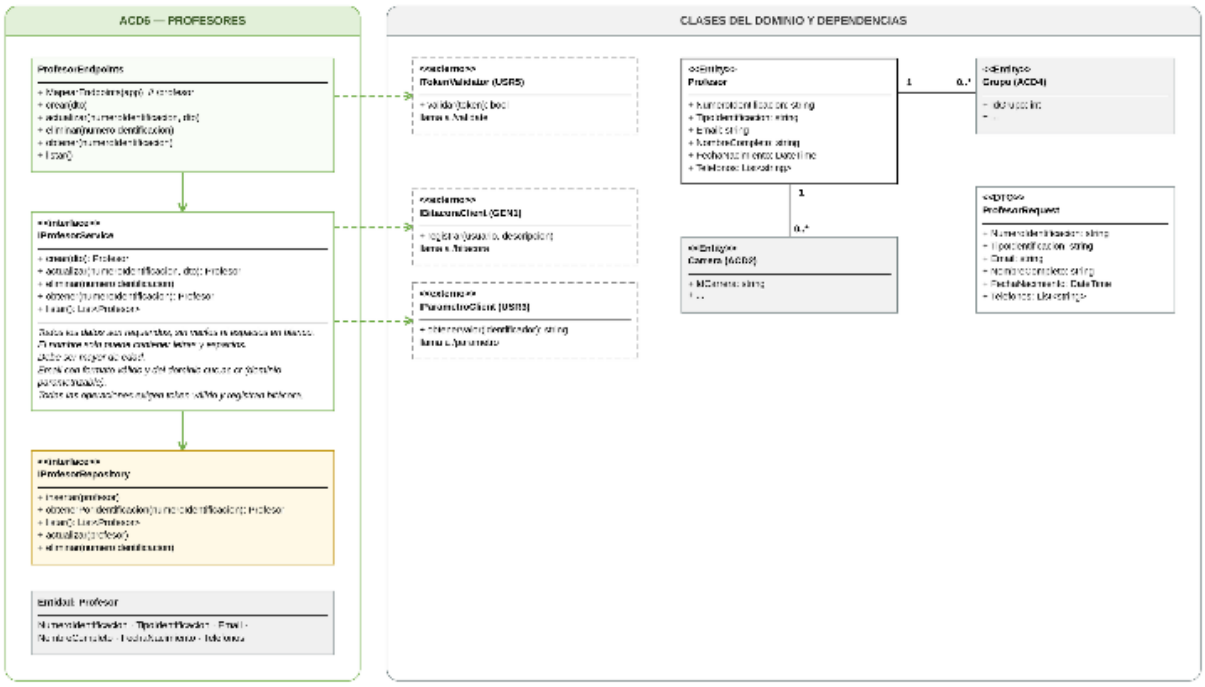
5.9 ACD4 – Administración de grupos



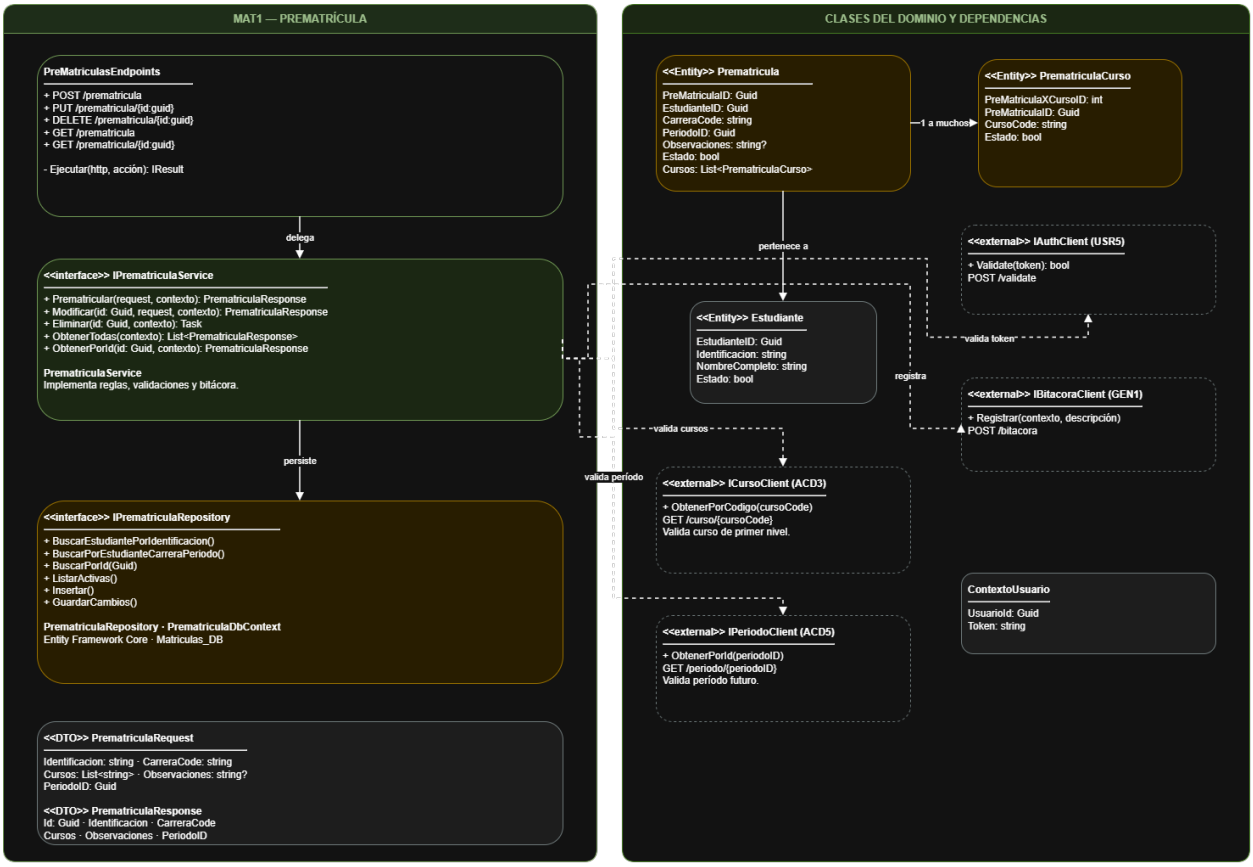
5.10 ACD5 – Administración de periodos



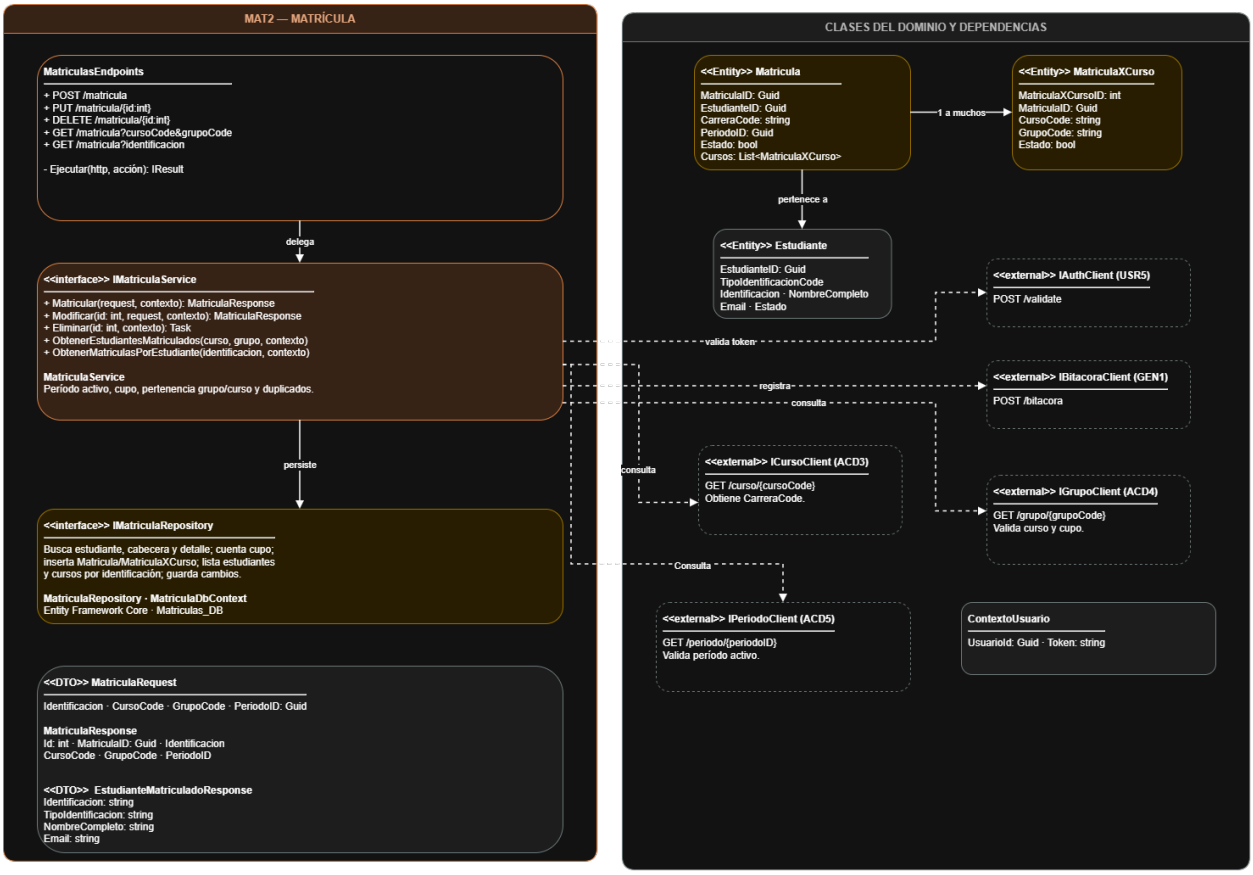
5.11 ACD6 – Administración de profesores



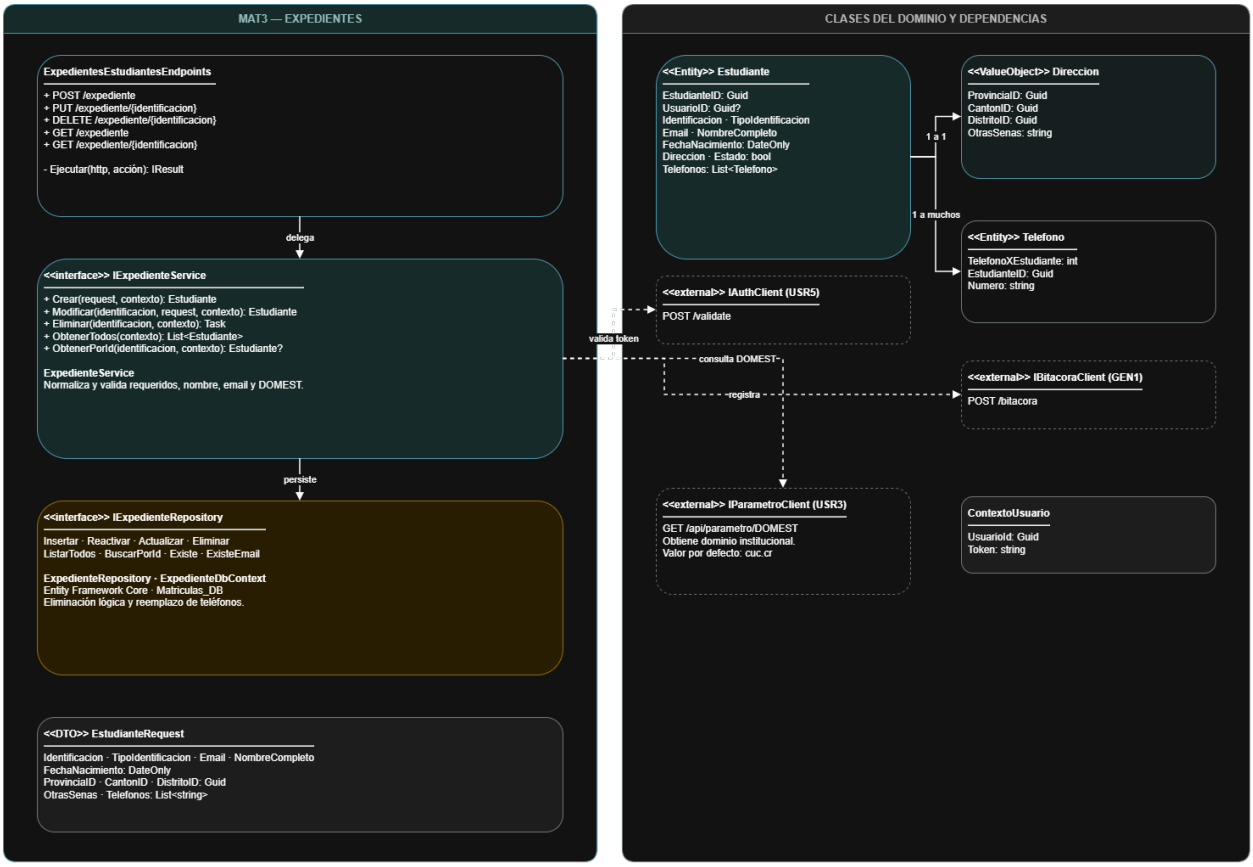
5.12 MAT1 – Administración de prematricula



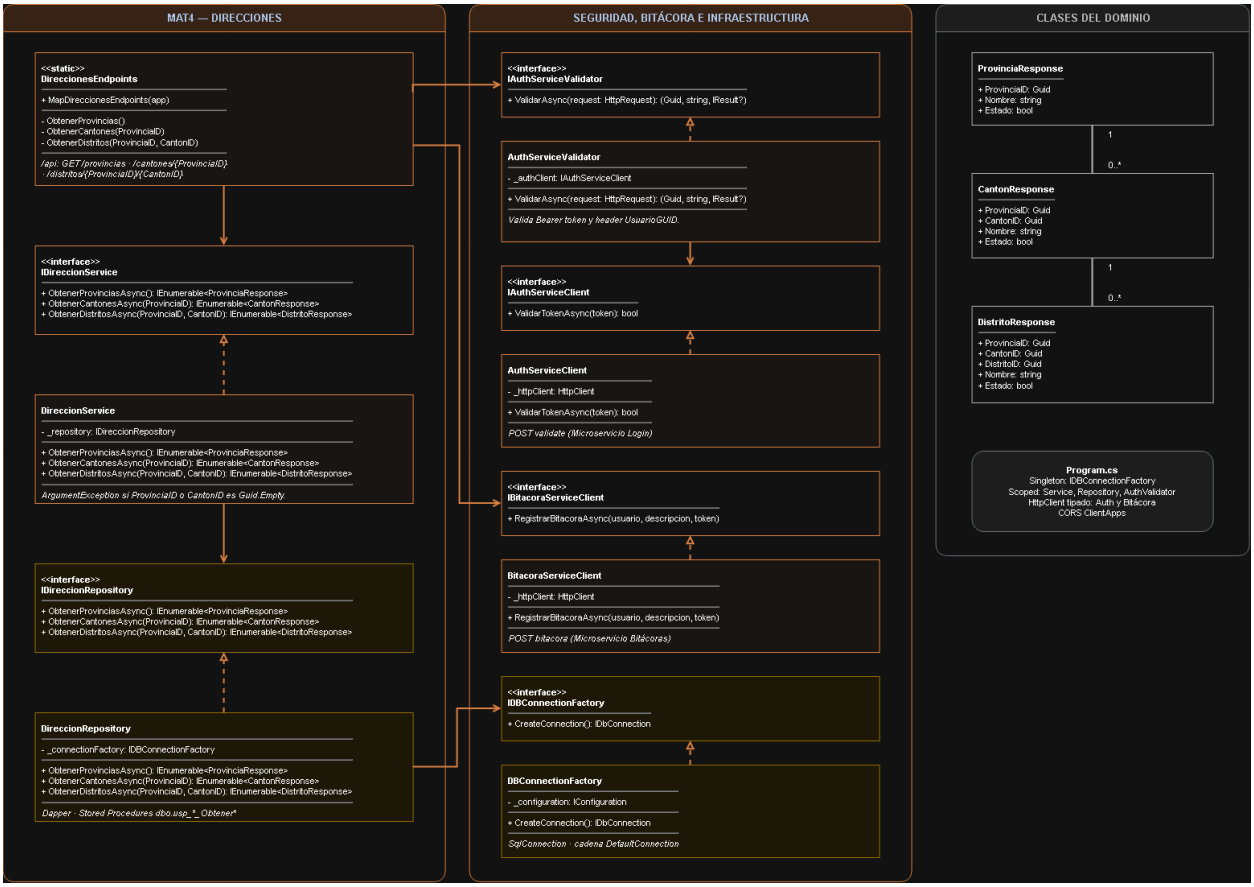
5.13 MAT2 – Administración de matrícula



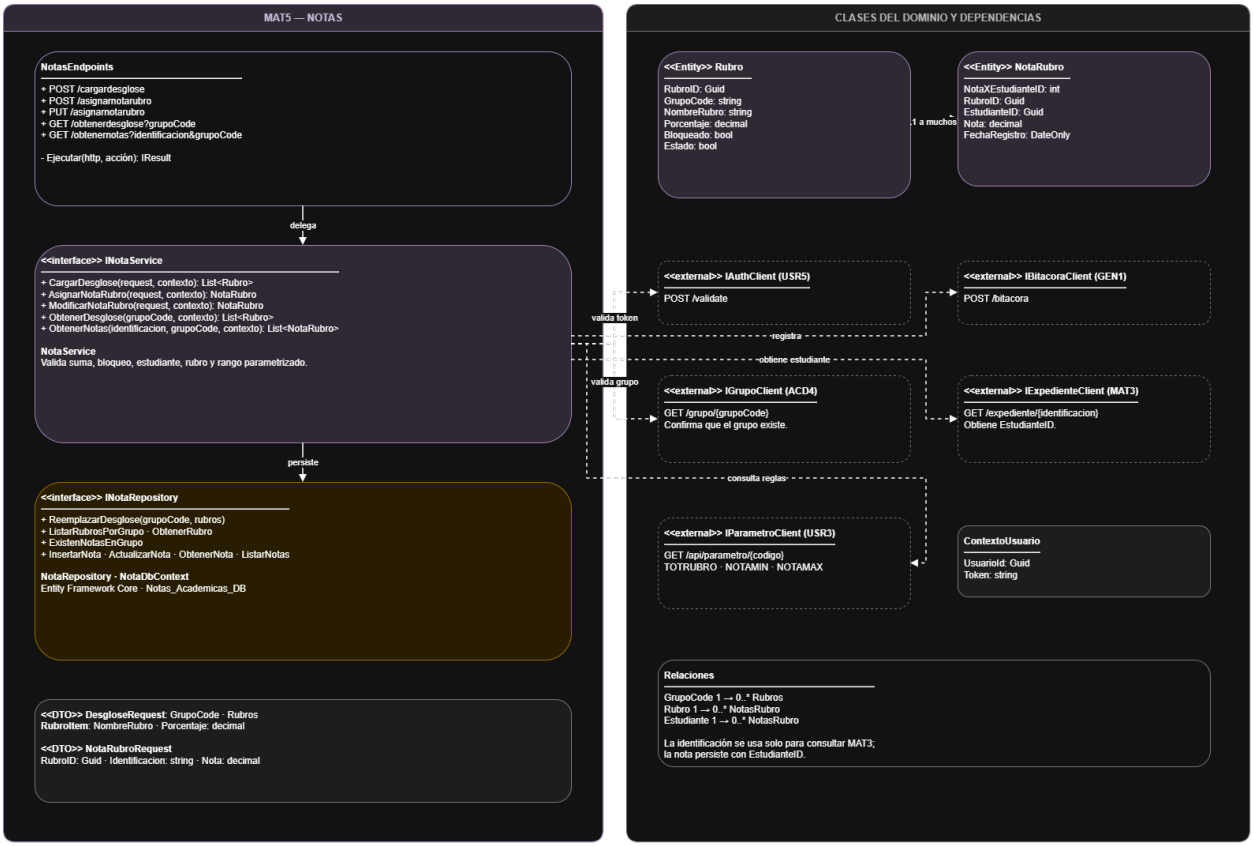
5.14 MAT3 – Administración de expedientes de estudiantes



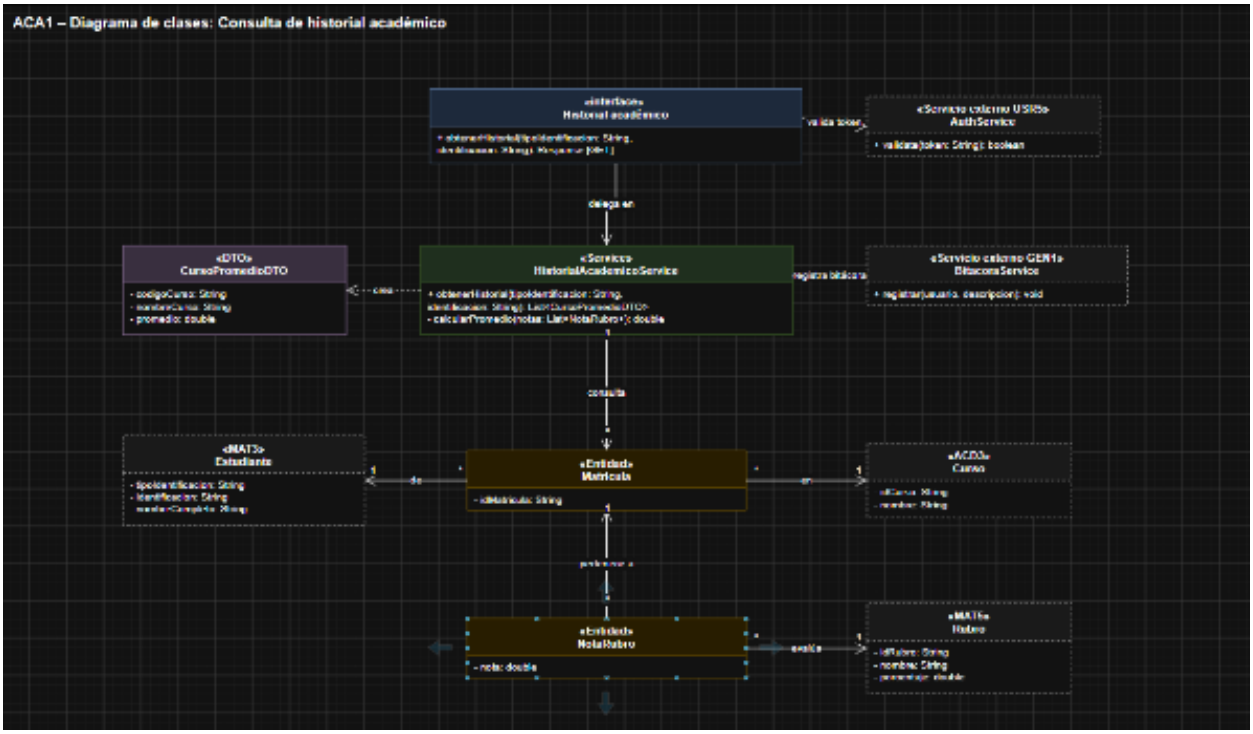
5.15 MAT4 – Administración de direcciones



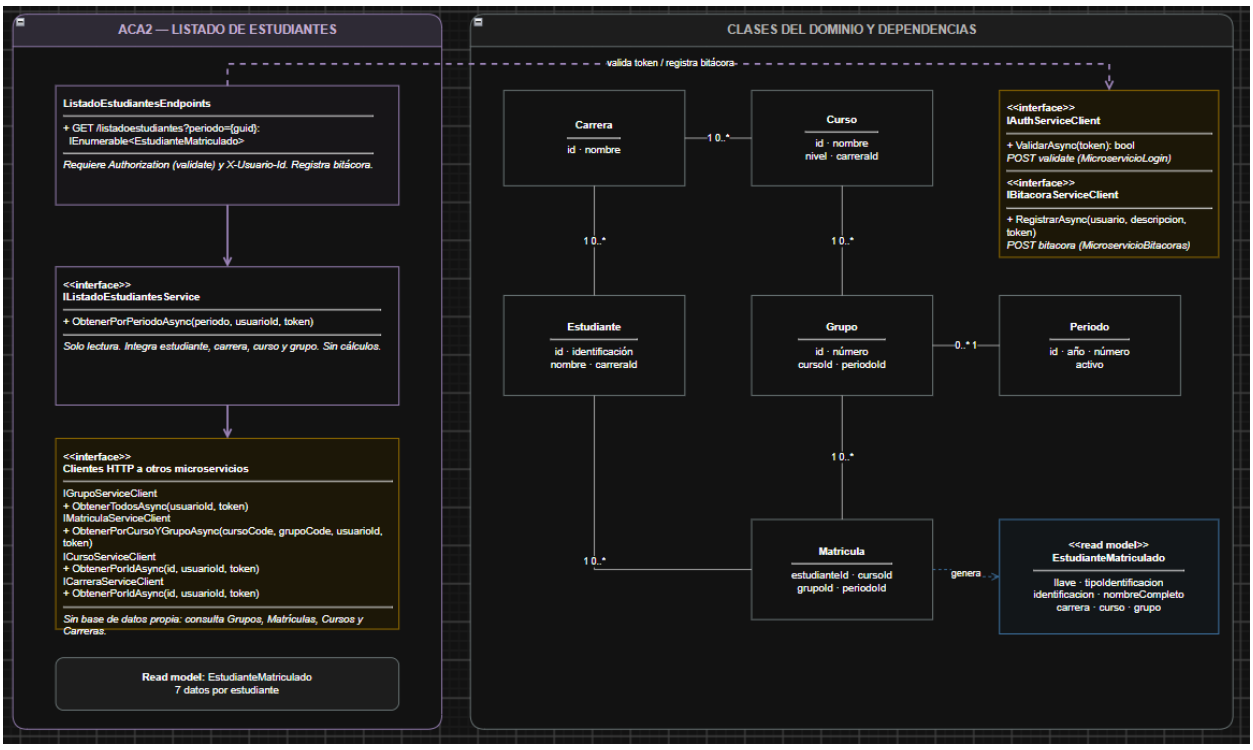
5.16 MAT5 – Administración de notas



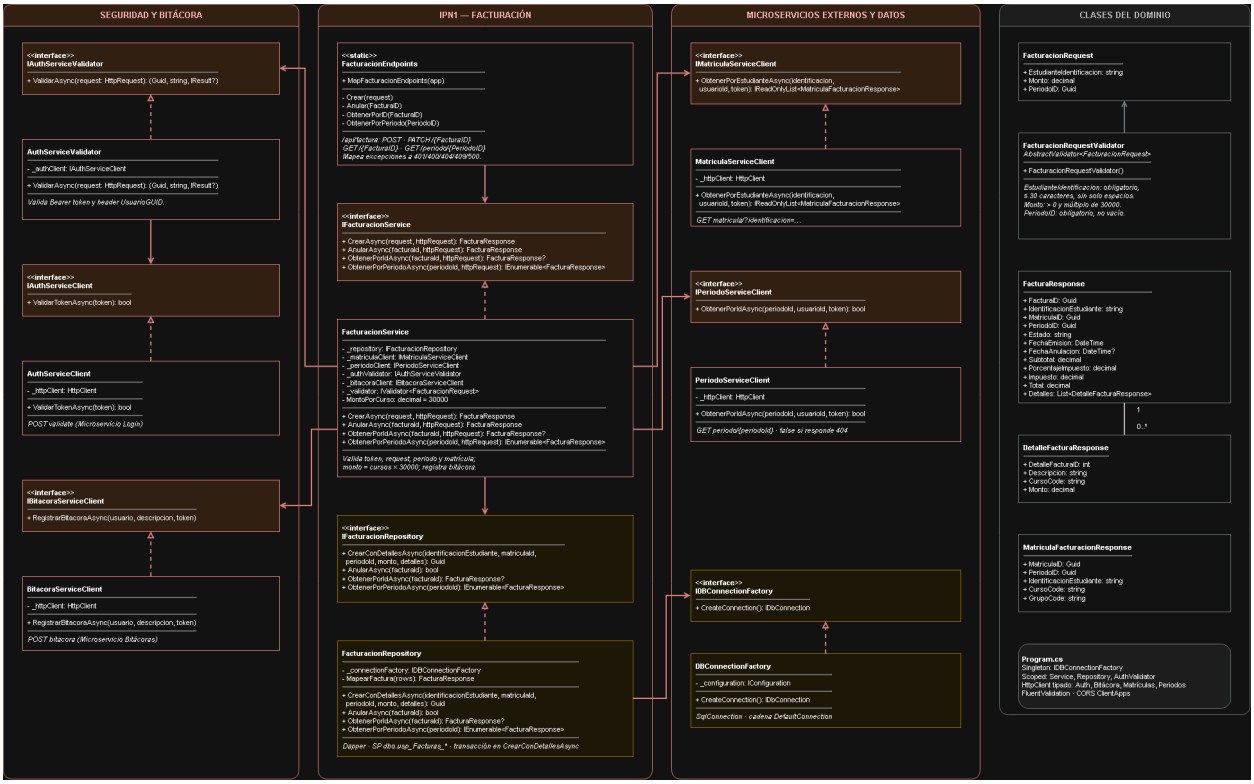
5.17 ACA1 – Consulta del historial académico



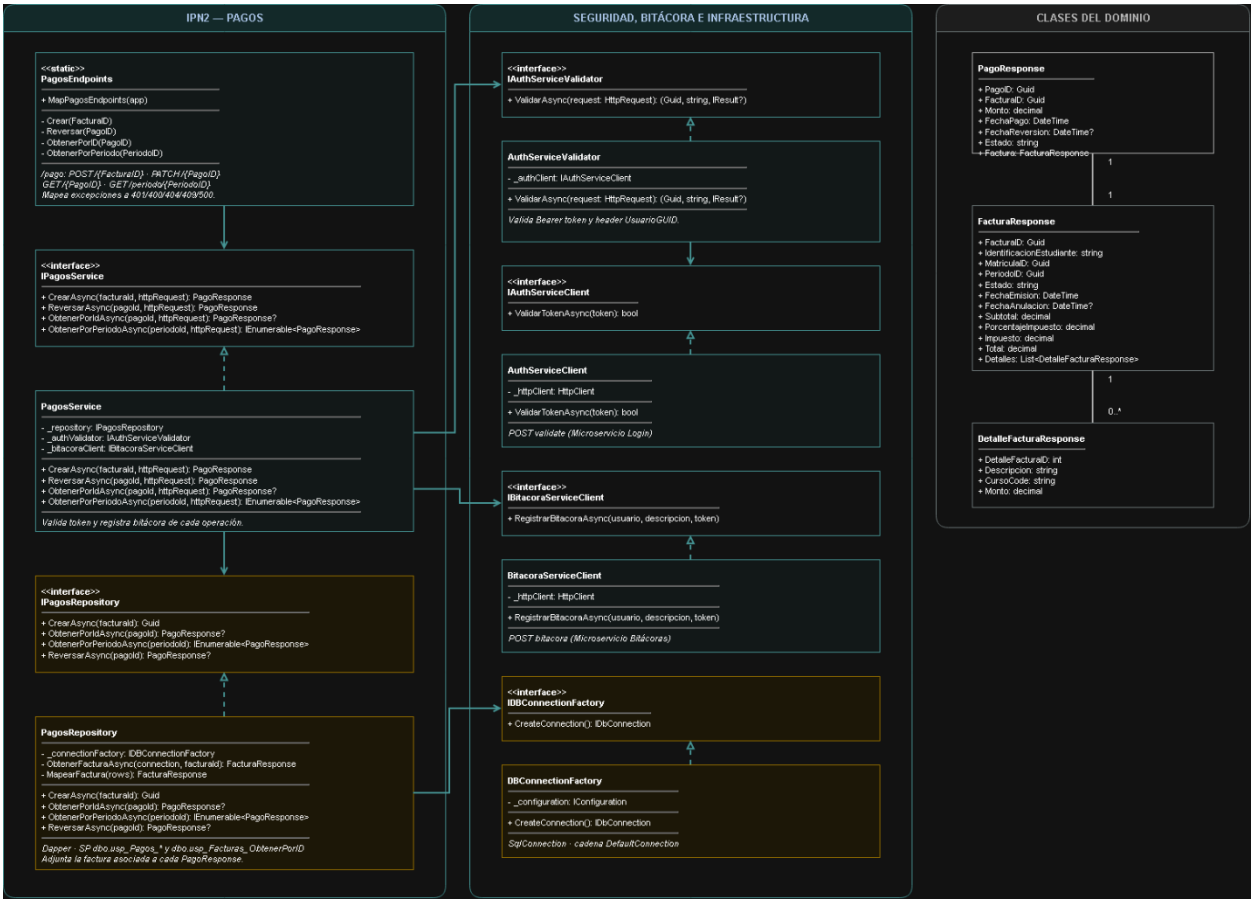
5.18 ACA2 – Consulta de listado de estudiantes



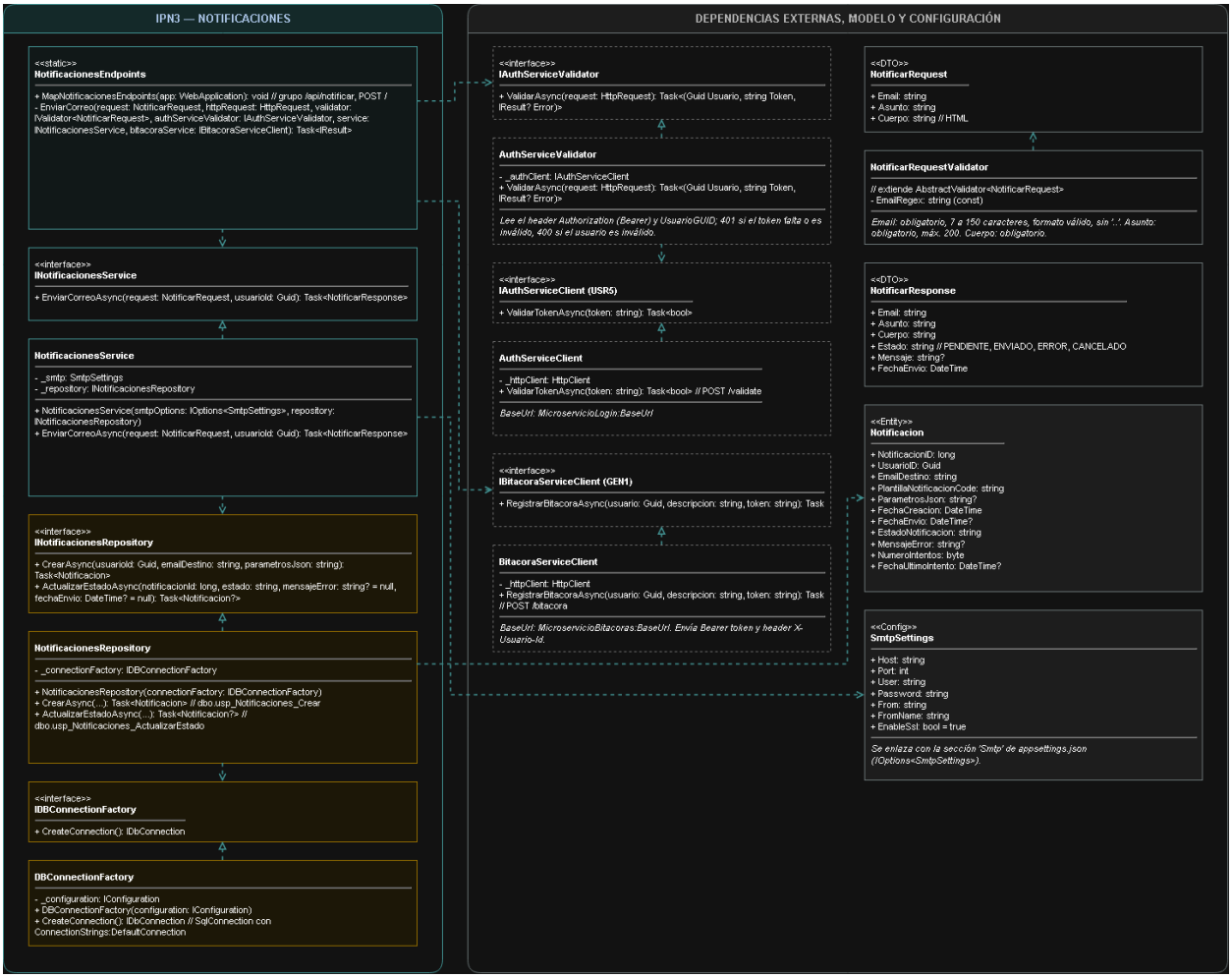
5.19 IPN1 – Administración de la facturación



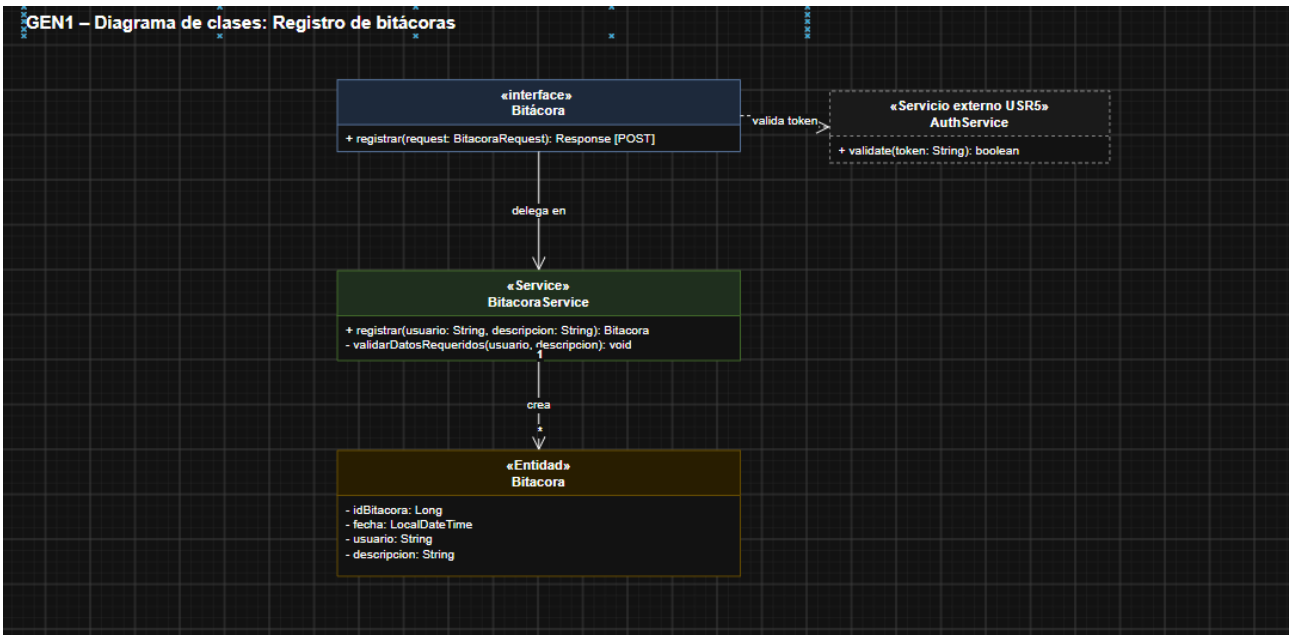
5.20 IPN2 – Administración de los pagos



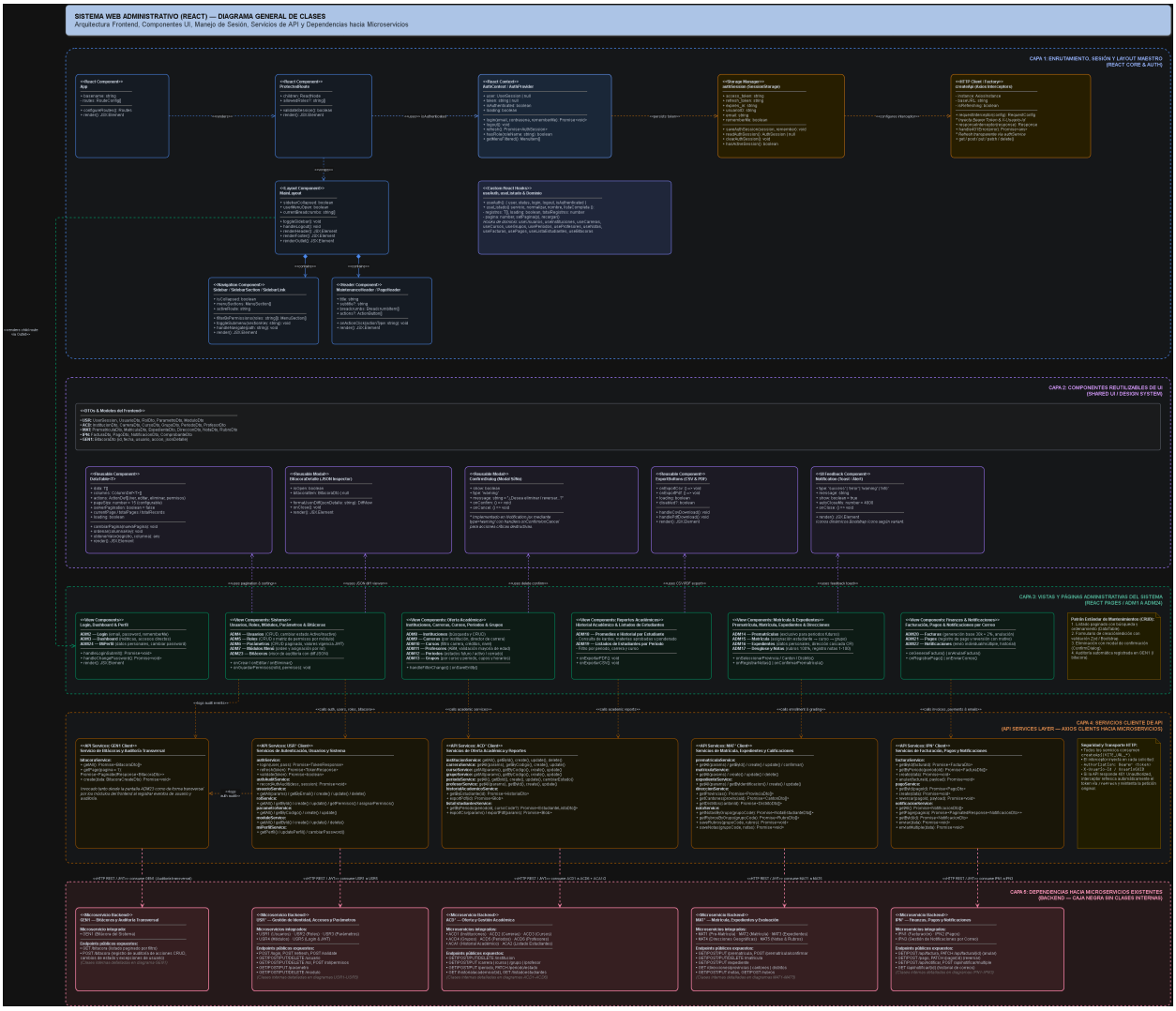
5.21 IPN3 – Gestión de notificaciones



5.22 GEN1 – Registro y consulta de bitácoras

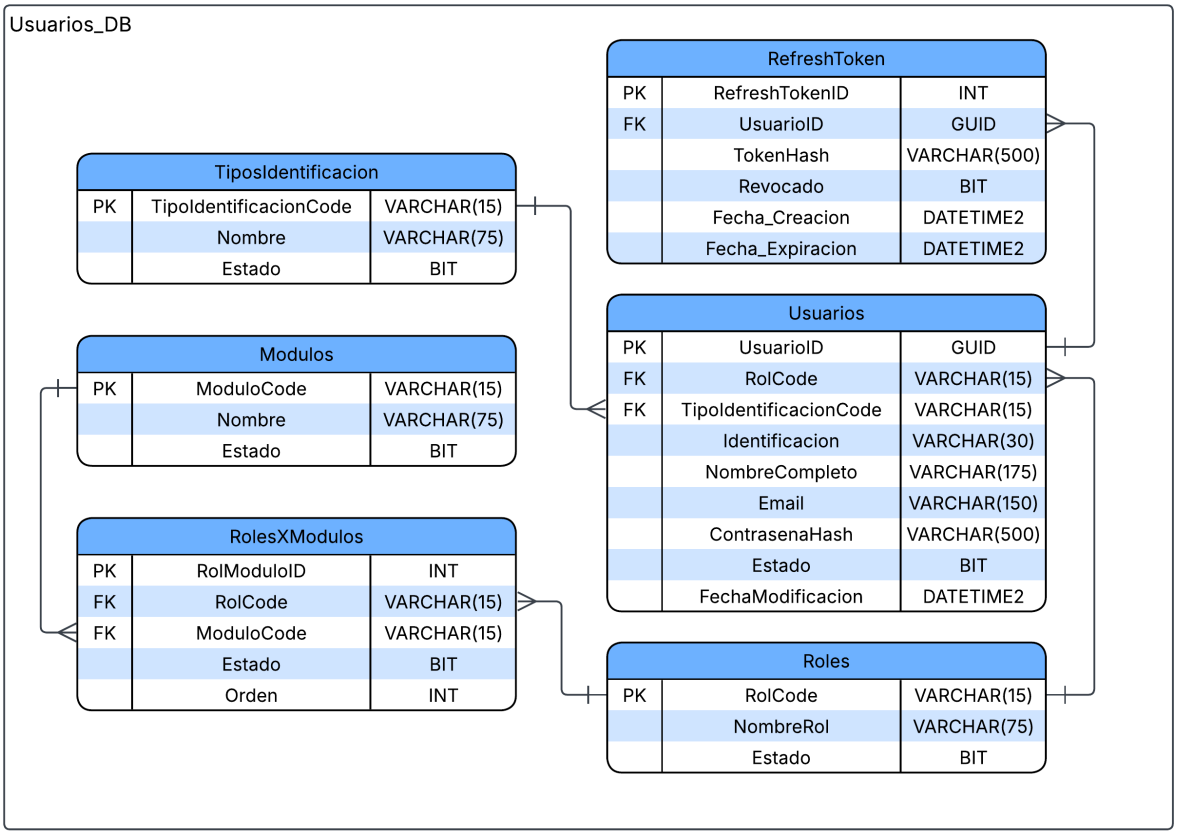

GEN1 – Diagrama de clases: Registro de bitácoras
«interface»
Bitácora
+ registrar(request: BitacoraRequest): Response [POST]
valida token
«Servicio externo USR5»
AuthService
+ validate(token: String): boolean
delega en
«Service»
BitacoraService
+ registrar(usuario: String, descripcion: String): Bitacora
- validarDatosRequeridos(usuario, descripcion): void
crea
«Entidad»
Bitacora
- idBitacora: Long
- fecha: LocalDateTime
- usuario: String
- descripcion: String

5.23 Website Administrativo

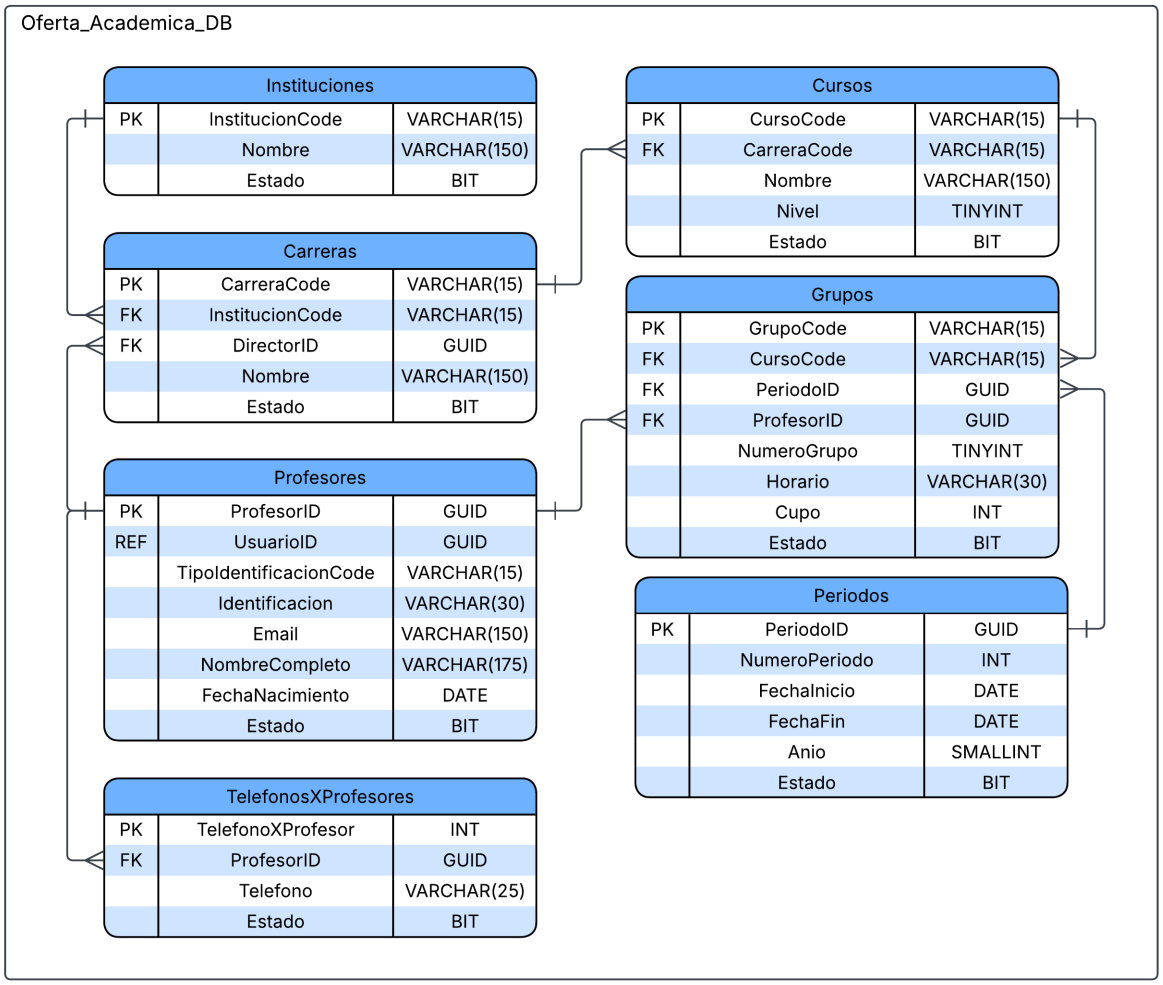


5. Diagramas de base de datos

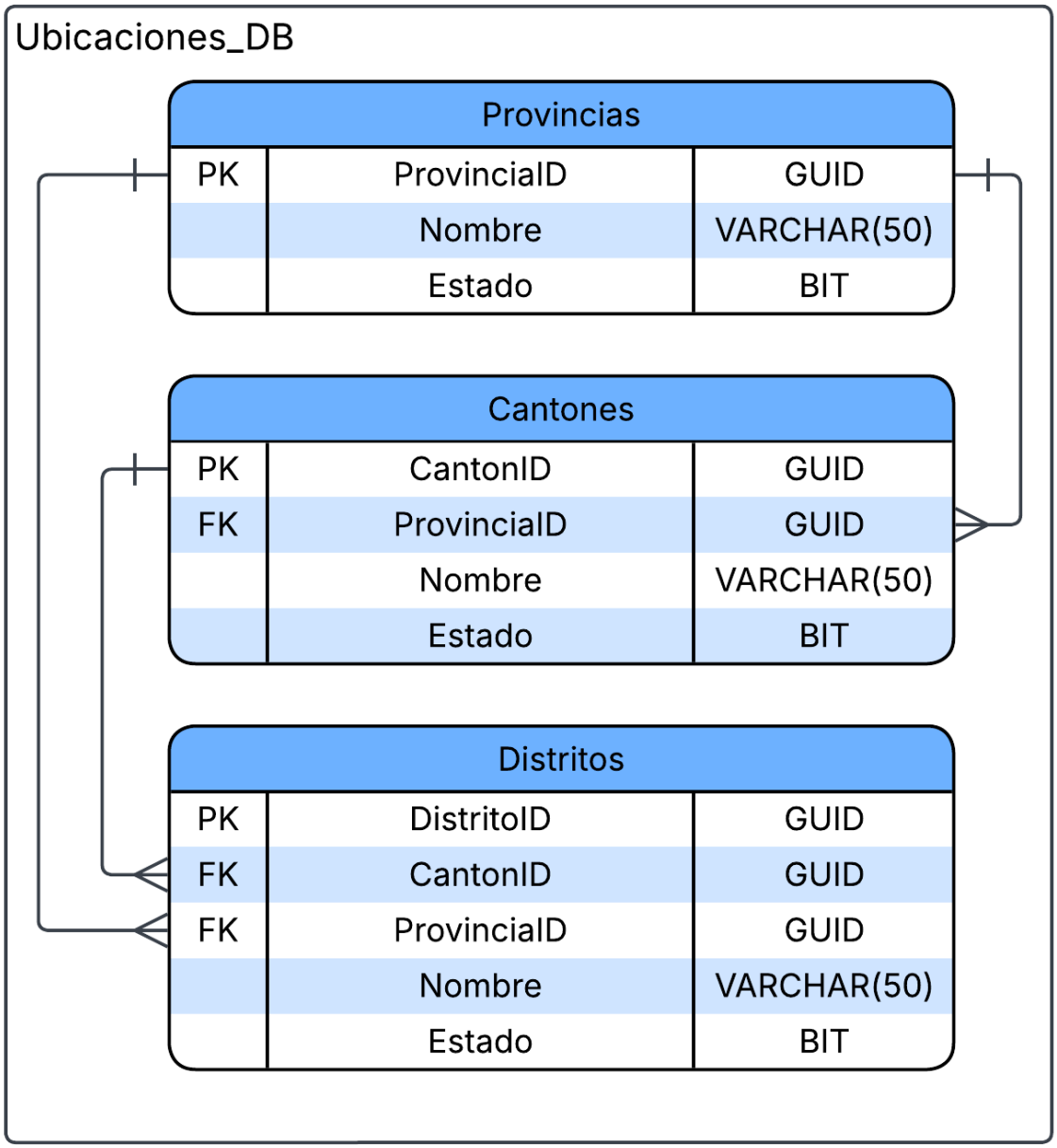
3.1 Usuarios_DB



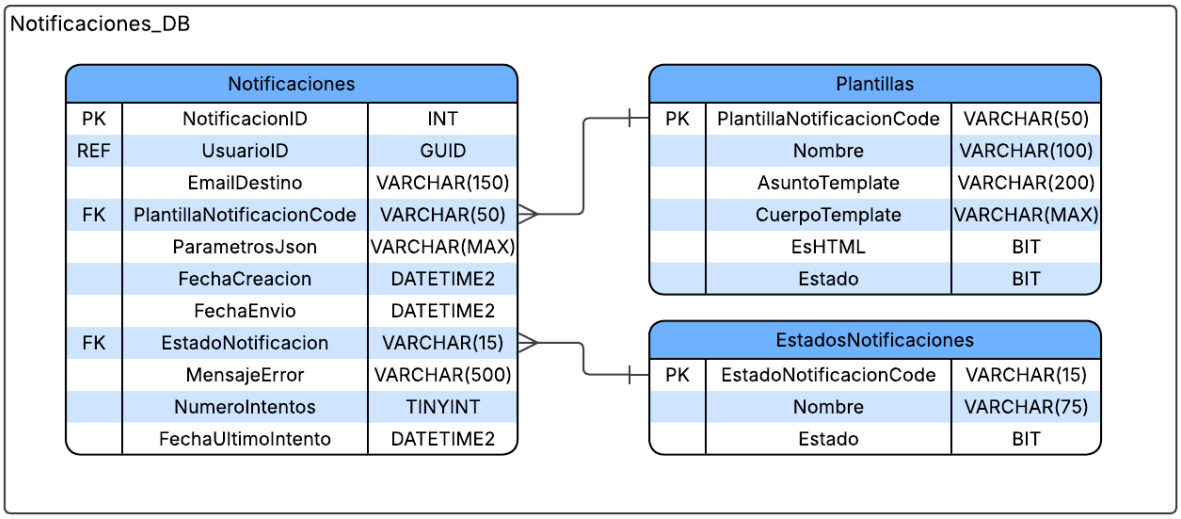
3.2 Oferta_Academica_DB



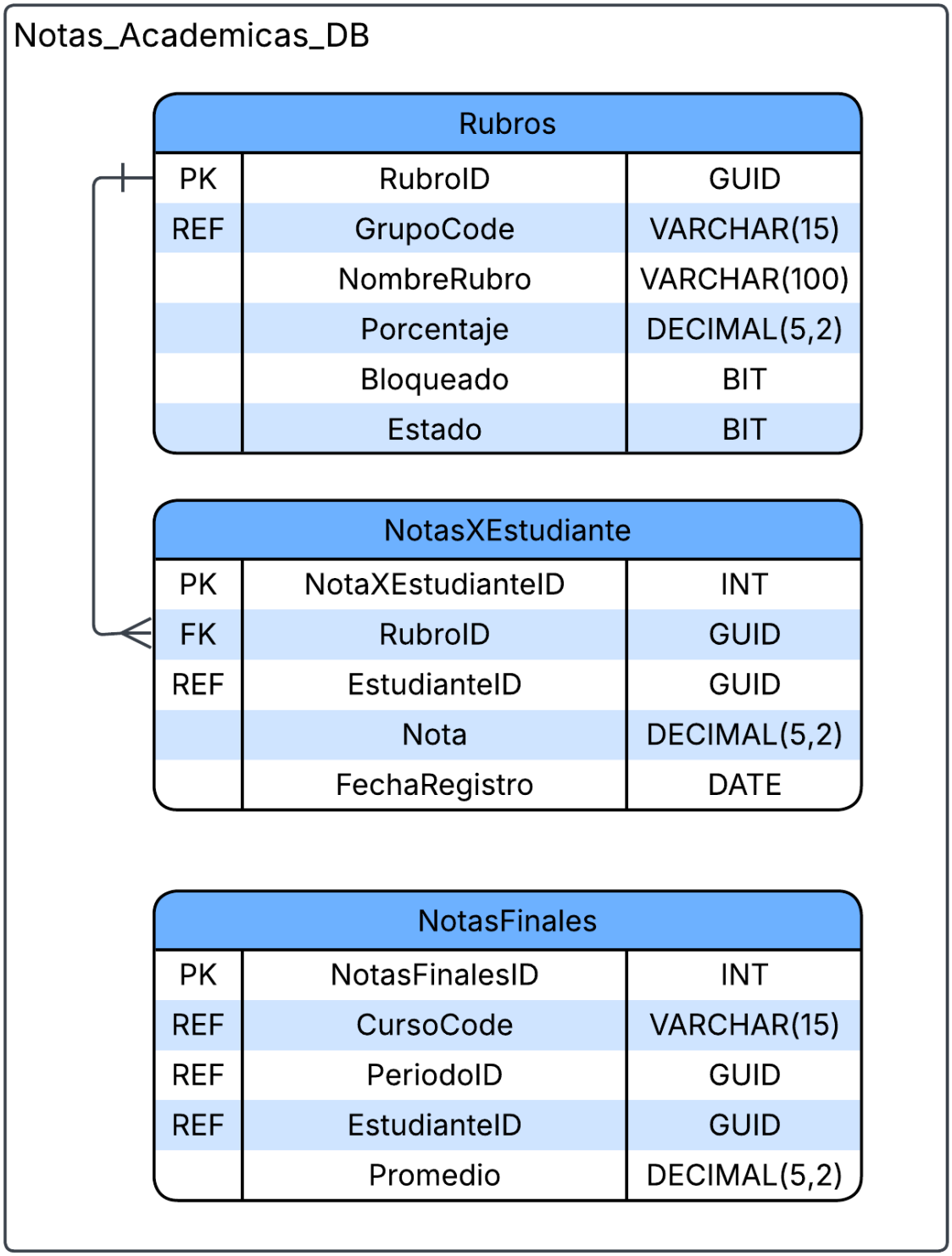
3.3 Ubicaciones_DB



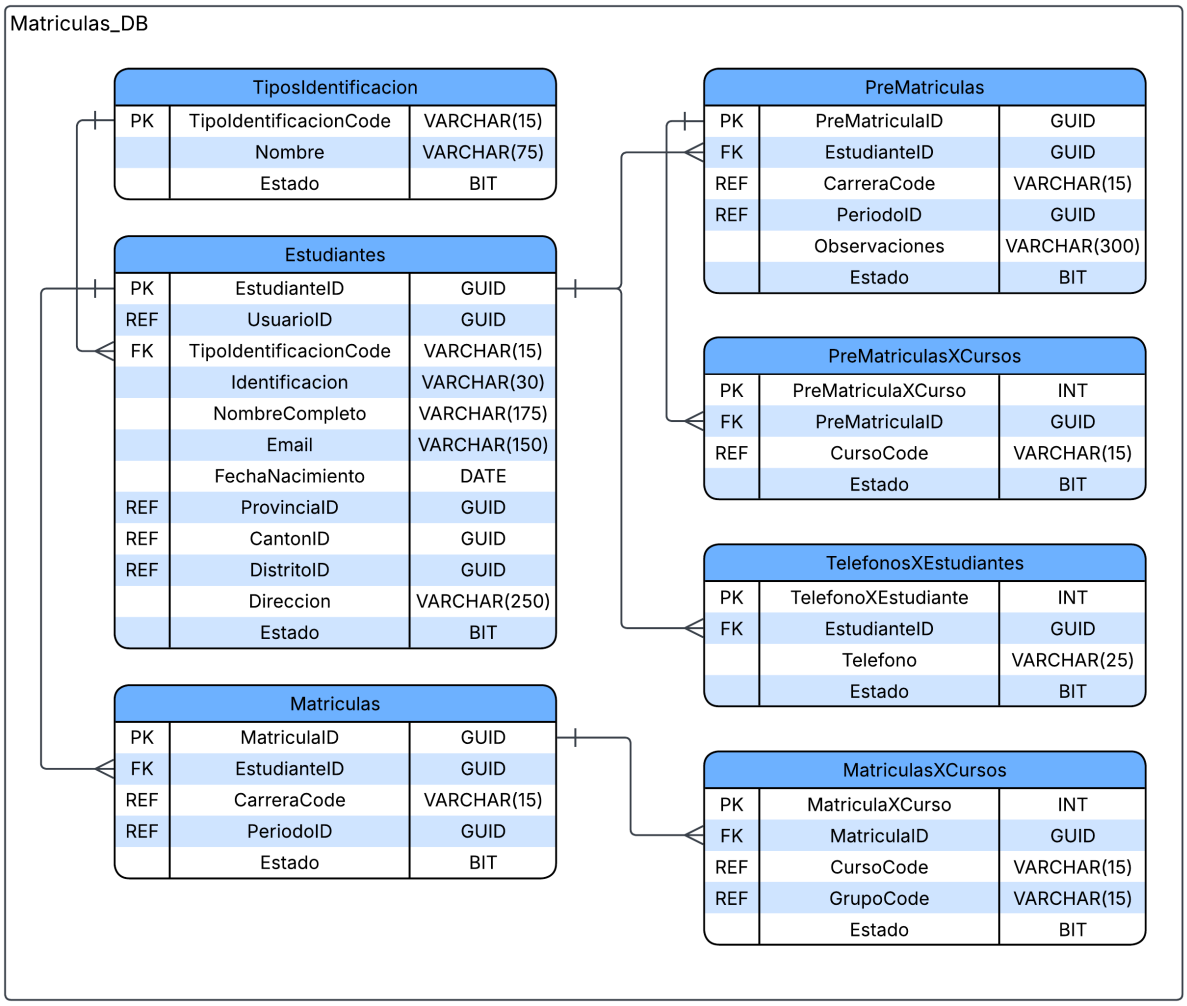
3.4 Notificaciones_DB



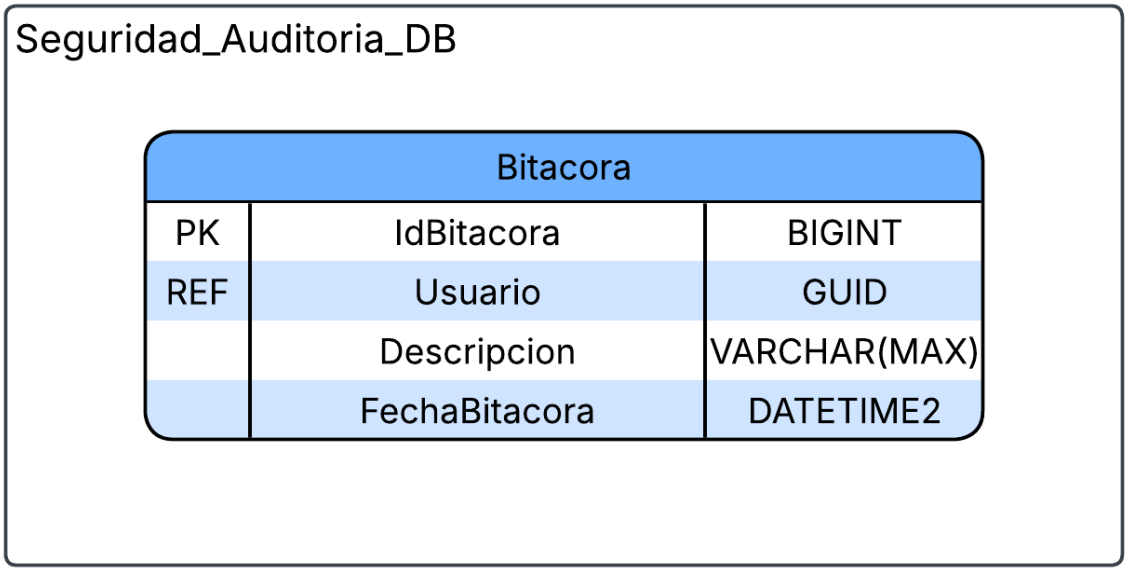
3.5 Notas_Academicas_DB



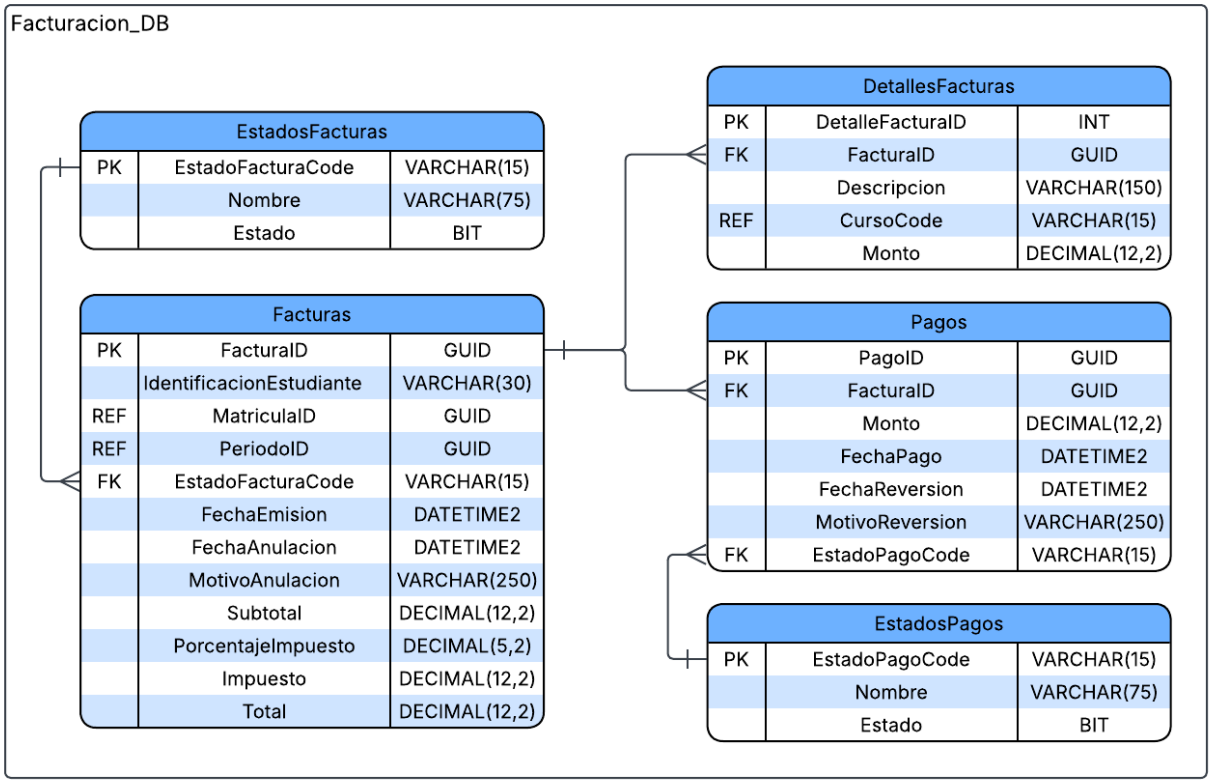
3.6 Matriculas_DB



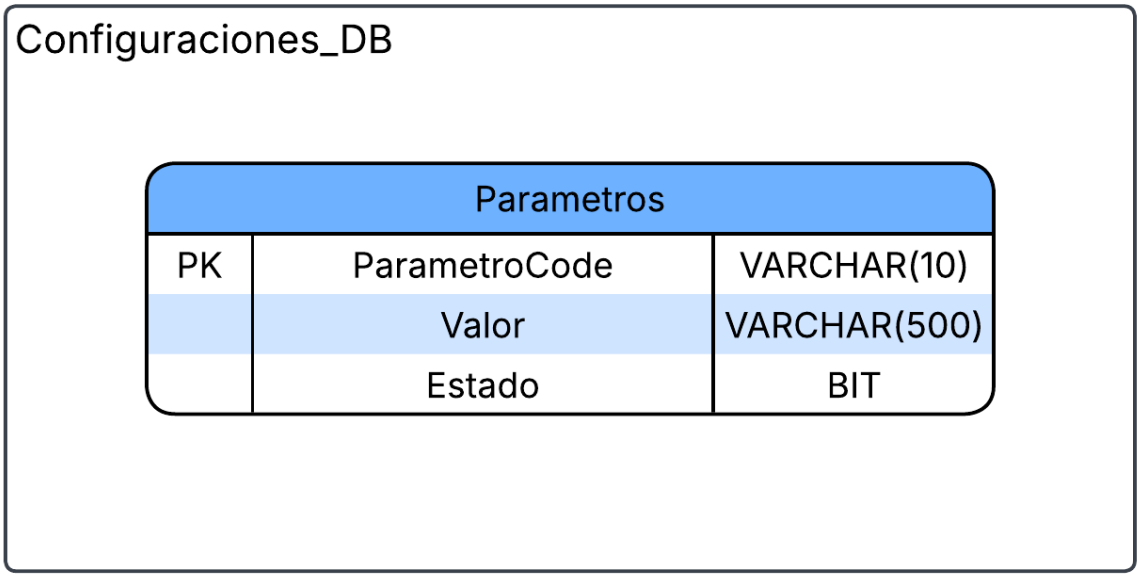
3.7 Seguridad_Auditoria_DB



3.8 Facturacion_DB



3.9 Configuraciones_DB



6. Conclusiones y recomendaciones

6.1 Conclusiones

Este documento de análisis y diseño del Proyecto MetaU (Propuesta Nuevo Avatar) evidencia que se ha completado satisfactoriamente la fase de análisis y diseño correspondiente al Primer Sprint de microservicios REST.

Se identificaron, analizaron y documentaron las 22 historias de usuario que conforman el alcance del sprint, cubriendo de manera integral los módulos de Usuarios y roles (USR1 a USR5), Oferta académica (ACD1 a ACD6), Matrícula (MAT1 a MAT5), Académico (ACA1 y ACA2), Integración de pagos y notificaciones (IPN1 a IPN3) y General (GEN1). Para cada una se definieron los criterios de aceptación, los endpoints REST, las validaciones de negocio, los requisitos de autenticación mediante tokens y el registro de bitácoras.

La arquitectura de microservicios propuesta permite que cada componente sea independiente, escalable y mantenible. El diseño de los diagramas de casos de uso, diagramas de clases y diagramas de base de datos (Usuarios_DB, Oferta_Academica_DB, Ubicaciones_DB, Notificaciones_DB, Notas_Academicas_DB, Matriculas_DB, Seguridad_Auditoria_DB, Facturacion_DB y Configuraciones_DB) proporciona una base sólida y coherente para la implementación.

El uso de estándares REST, autenticación basada en JWT con refresh tokens, validaciones de dominio (emails @cuc.cr y @cuc.ac.cr), parametrización de valores críticos y el registro sistemático de bitácoras (GEN1) garantiza la seguridad, la trazabilidad y la flexibilidad del sistema. Asimismo, la organización del trabajo bajo el marco Scrum y el uso de control de versiones con Git/GitHub (ramas Main, Develop y Feature) facilitan el trabajo colaborativo y la integración continua del código.

El análisis y diseño realizado cubre de forma completa los requisitos del primer sprint y establece las bases técnicas y funcionales necesarias para avanzar a la fase de implementación de los microservicios.

6.2 Recomendaciones

- Priorizar en los siguientes sprints la implementación de los microservicios siguiendo estrictamente los diagramas de clases y de base de datos definidos, utilizando ASP.NET Core Minimal APIs y Dapper sobre SQL Server.
- Implementar de forma centralizada el middleware de validación de tokens (endpoint /validate de USR5) y el servicio de registro de bitácoras (GEN1) para que todos los demás servicios los consuman de manera consistente.
- Definir y documentar una estrategia clara de manejo de errores, códigos de estado HTTP y respuestas estandarizadas en todos los endpoints, incluyendo los mensajes específicos indicados en las historias de usuario (por ejemplo, “Usuario y/o contraseña incorrectos”).

- Establecer pruebas unitarias, de integración y de contrato (por ejemplo con Postman o tests automatizados) para cada microservicio antes de integrarlos, con especial atención a las validaciones de negocio (dominios de correo, mayor de edad, rangos de nivel, etc.).
- Parametrizar desde el inicio todos los valores indicados como configurables (tiempos de expiración de tokens, dominios de correo permitidos, tipos de identificación, etc.) mediante el microservicio de parámetros (USR3).
- Mantener actualizada la documentación técnica (diagramas y este documento) conforme avance la implementación y se realicen ajustes derivados de las revisiones del producto o de las pruebas.
- Considerar la incorporación de un API Gateway y de mecanismos de observabilidad (logs centralizados, métricas y trazas) en sprints posteriores para facilitar el monitoreo y la operación del sistema en producción.
- Realizar revisiones de código periódicas y sesiones de refinamiento del backlog para asegurar que las historias de usuario pendientes o las mejoras identificadas se prioricen correctamente.

7. Bibliografía

- González Solano, G. (2026). *Primer Sprint – Proyecto Nuevo Avatar V1*. Colegio Universitario de Cartago.
- Microsoft. (2025). *Documentación de ASP.NET Core*. https://learn.microsoft.com/es-es/aspnet/core/
- Microsoft. (2025). *Minimal APIs en ASP.NET Core*. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/minimal-apis
- Microsoft. (2025). *Entity Framework Core*. https://learn.microsoft.com/es-es/ef/core/
- Newman, S. (2021). *Building Microservices* (2.ª ed.). O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Sommerville, I. (2016). *Software Engineering* (10.ª ed.). Pearson.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Tesis doctoral). University of California, Irvine. (Definición de REST).
- RFC 7519. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7519
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7.ª ed.). Pearson.